

Resource Coordination Platform for Community Resilience Software Architecture Document

Version 1.0

MENTOR: MR. ISURU SAMARANAYAKE

Kumarasinghe H.R.P.R. – 230354T

Lakmal H.M.P. – 230361L

Lakshika G. – 230370M

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

Revision History

Date	Version	Description	Author
09/08/2026	1.0	Architecture document	Group 14

Member	Section
Kumarasinghe H.R.P.R. – 230354T	6, 7, 8, 9, 10, 11
Lakmal H.M.P. – 230361L	4, 5
Lakshika G – 230370M	1, 2, 3

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

Table of Contents

1.	Introduction.....	4
1.1	Purpose	4
1.2	Scope	4
1.3	Definitions, Acronyms, and Abbreviations	4
1.4	References	5
1.5	Overview	5
2.	Architectural Representation.....	5
2.1	Overall System Architecture.....	7
2.2	Component View	7
3.	Architectural Goals and Constraints	8
3.1	Architectural Goals.....	8
3.2	Architectural Constraints	8
3.3	Quality Attributes	8
4.	Use-Case View	9
4.1	Use-Case Realizations	11
5.	Logical View	25
5.1	Overview	25
5.2	Architecturally Significant Design Packages	26
6.	Process View	28
7.	Deployment View	59
8.	Implementation View	59
8.1	Overview	59
8.2	Layers	59
9.	Data View (optional)	62
10.	Size and Performance	63
11.	Quality	63
12.	References	63

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

Software Architecture Document

1. Introduction

1.1 Purpose

The purpose of this Software Architecture Document (SAD) is to describe the overall software architecture of the Resource Coordination Platform for Community Resilience, a distributed web and mobile application designed to coordinate disaster relief operations efficiently.

This document presents the system's architectural design, major components, interactions between components, architectural constraints, quality attributes, and key design decisions. It serves as a technical reference for software developers, system architects, testers, project managers, and other stakeholders throughout the software development lifecycle.

1.2 Scope

The Resource Coordination Platform is a microservices-based system that supports the coordination of disaster response activities by connecting community members, volunteers, donors, organizations, and administrators through a centralized platform.

The architecture described in this document includes:

- Web application developed using React
- Mobile application developed using Flutter
- Kong API Gateway for centralized API routing
- User, Organization, Request, Resource, Volunteer, Task, and Notification microservices
- Identity and Access Management (IAM) framework for authentication and authorization
- Apache Kafka for asynchronous event-driven communication
- PostgreSQL (Supabase) as the centralized relational database
- Docker-based deployment environment
- RESTful APIs and Swagger/OpenAPI documentation

The document focuses only on the architectural design of the system and does not describe detailed implementation or testing procedures.

1.3 Definitions, Acronyms, and Abbreviations

Abbreviation	Definition
API	Application Programming Interface
Docker	Containerization platform used to package and deploy applications
IAM	Identity and Access Management
JWT	JSON Web Token
Kafka	Apache Kafka – Distributed event streaming platform
Kong	API Gateway responsible for routing and API management
PostgreSQL	Open-source relational database management system
RBAC	Role-Based Access Control
REST	Representational State Transfer
SAD	Software Architecture Document
SRS	Software Requirements Specification
SSO	Single Sign-On

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

Swagger	OpenAPI-based API documentation tool
UUID	Universally Unique Identifier

1.4 References

- [1] Tool used for diagrams
Diagrams: <https://app.diagrams.net/>
- [2] Microservices
<https://www.geeksforgeeks.org/system-design/microservices/>
- [3] Apache Software Foundation, *Apache Kafka Documentation*.
<https://kafka.apache.org/documentation/>
- [4] PostgreSQL Global Development Group, *PostgreSQL Documentation*.
<https://www.postgresql.org/docs/>
- [5] Express.js Documentation.
<https://expressjs.com/>
- [6] Node.js Documentation.
<https://nodejs.org/docs/latest/api/>
- [7] React Documentation.
<https://react.dev/>
- [8] Flutter Documentation.
<https://docs.flutter.dev/>
- [9] OpenAPI Initiative, *OpenAPI Specification*.
<https://swagger.io/specification/>
- [10] Kong Inc., *Kong Gateway Documentation*.
<https://docs.konghq.com/gateway/latest/>
- [11] Identity and Access Management
<https://firebase.google.com/>
- [12] JSON Web Token (JWT) RFC 7519.
<https://datatracker.ietf.org/doc/html/rfc7519>

1.5 Overview

The remainder of this document is organized as follows:

- Chapter 2: describes the overall software architecture and architectural representation.
- Chapter 3: presents the architectural goals, design principles, and constraints.
- Chapter 4: describes the use-case view.
- Chapter 5: presents the logical design of the system.
- Chapter 6: explains the process view.
- Chapter 7: describes the deployment architecture.
- Chapter 8: explains the implementation architecture.
- Chapter 9: presents the data view.
- Chapter 10: discusses performance considerations.
- Chapter 11: describes quality attributes of the architecture.

2. Architectural Representation

The Resource Coordination Platform uses a microservices architecture to offer a flexible and maintainable solution for coordinating disaster resources. It consists of independent services, each handling specific functions like user management, organization management, resource management, request management, volunteer management, task management, and notification delivery.

The platform has two client applications: a web application built with React and a mobile application developed with Flutter. Both applications connect to the backend through the Kong API Gateway, which serves as the main access point for all client requests. The API Gateway manages request routing, forwards authentication, and oversees API management. The chosen Identity and Access Management (IAM) framework takes care of authentication and authorization. It verifies users and issues JSON Web Tokens

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

(JWTs) for secure access to protected resources. Role-Based Access Control (RBAC) ensures users only access services and resources allowed by their roles. The backend consists of the following independent microservices,

- User Service
- Organization Service
- Resource Service
- Request Service
- Volunteer Service
- Task Service
- Notification Service

Each microservice exposes RESTful APIs and performs a dedicated business function. This separation enables independent development, deployment, testing, and maintenance while reducing coupling between system components.

The platform uses PostgreSQL (Supabase) as the centralized relational database for persistent data storage. All services access the database according to their functional responsibilities while maintaining data consistency through well-defined service boundaries.

For asynchronous communication, the architecture utilizes Apache Kafka as the event streaming platform. Business events generated by services such as the Request Service, Resource Service, Volunteer Service, and Task Service are published to Kafka topics. The Notification Service subscribes to these topics and generates appropriate notifications for affected users without introducing direct dependencies between services.

Containerization using Docker provides a consistent runtime environment for all backend services and simplifies deployment across development, testing, and production environments.

Overall, the selected microservices architecture improves scalability, maintainability, extensibility, and fault isolation, making it well suited for a disaster response platform that must support multiple user roles and concurrent operations.

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

2.1 Overall System Architecture

The overall system architecture illustrates the high-level structure of the Resource Coordination Platform and the interactions among the client applications, API Gateway, Identity and Access Management framework, backend microservices, Apache Kafka, and the PostgreSQL database.

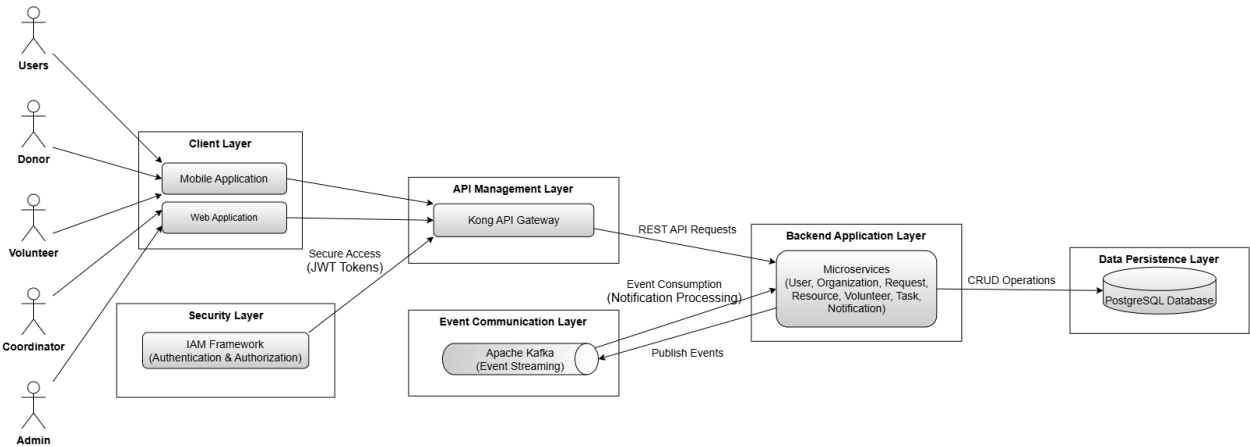


Figure 1 Overall System Architecture of the Resource Coordination Platform.

2.2 Component View

The Component Diagram presents the logical organization of the major software components and their relationships. It illustrates how the web and mobile clients interact with the Kong API Gateway, how requests are routed to the appropriate backend microservices, how authentication is managed through the IAM framework, and how the services communicate with the PostgreSQL database and Apache Kafka.

This view highlights the modular nature of the system by showing each microservice as an independent component responsible for a specific business capability. The diagram also demonstrates the interaction between the Notification Service and Apache Kafka for event-driven communication.

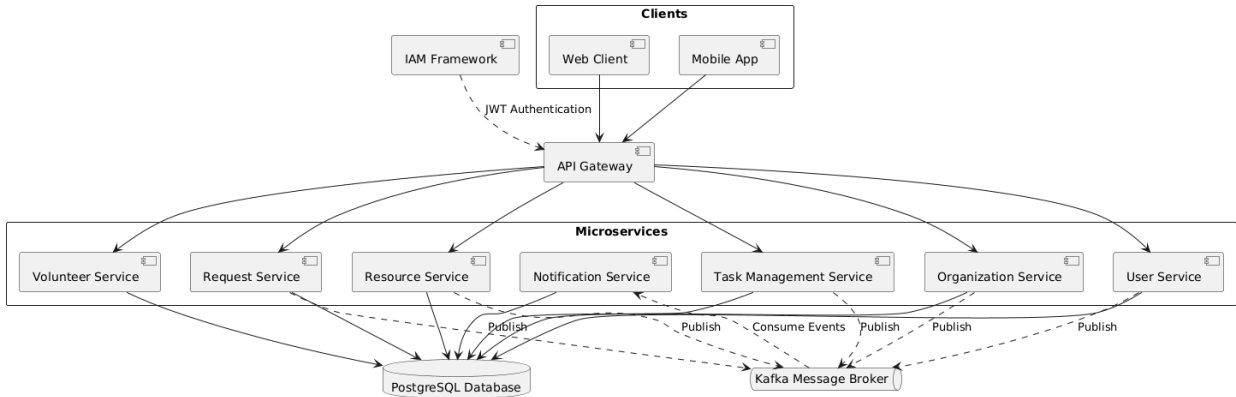


Figure 2 Component Diagram of the Resource Coordination Platform.

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

3. Architectural Goals and Constraints

3.1 Architectural Goals

The architecture of the Resource Coordination Platform is designed to achieve the following goals:

- **Scalability**
The system shall support increasing numbers of users, organizations, volunteers, and disaster requests without significant degradation in performance.
- **Maintainability**
Each microservice shall be independently developed, tested, deployed, and maintained to simplify future enhancements and bug fixes.
- **Security**
The architecture shall ensure secure authentication, authorization, and data transmission through the IAM framework, JWT-based authentication, HTTPS communication, and role-based access control.
- **Reliability**
The system shall continue operating during partial failures by isolating failures within individual services and maintaining data consistency.
- **Performance**
The architecture shall provide low response times for user interactions while efficiently handling concurrent requests and background event processing.
- **Modularity**
Each microservice shall encapsulate a specific business capability, reducing dependencies between services and promoting loose coupling.
- **Extensibility**
The architecture shall support the integration of additional services and future system enhancements with minimal impact on existing components.

3.2 Architectural Constraints

The architecture shall comply with the following constraints:

- The web application shall be developed using React.
- The mobile application shall be developed using Flutter.
- Backend services shall be implemented using Node.js and Express.js.
- The system shall expose functionality through RESTful APIs.
- Kong API Gateway shall manage API routing and request forwarding.
- Authentication and authorization shall be provided by the selected IAM framework.
- Apache Kafka shall be used for asynchronous event-driven communication.
- PostgreSQL (Supabase) shall serve as the centralized relational database.
- Docker shall be used to containerize all backend services.
- API documentation shall be generated using Swagger/OpenAPI.

3.3 Quality Attributes

The architecture is designed to satisfy the following quality attributes:

- **Performance:** Efficient request processing and low response times for critical operations.
- **Scalability:** Independent scaling of microservices based on workload.
- **Availability:** Continuous operation through service isolation and reliable communication mechanisms.
- **Security:** Secure access control, encrypted communication, and authenticated API access.

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

- **Reliability:** Consistent system behavior and dependable disaster response operations.
- **Maintainability:** Modular architecture supporting independent updates and simplified maintenance.
- **Extensibility:** Easy integration of new services and future functionality.
- **Usability:** Consistent and intuitive interfaces across web and mobile applications.

Quality Attribute	Architectural Approach
Security	IAM, JWT, HTTPS
Scalability	Microservices
Reliability	Kafka Event Processing
Maintainability	Independent Services
Performance	REST APIs + Database Indexing

4. Use-Case View

This section details the primary use cases that represent the central functionality of the Resource Coordination Platform for Community Resilience. The system supports distinct interactions for five primary actors, enforcing strict Role-Based Access Control (RBAC) boundaries.

Primary Actors:

- **Community Member:** Submits requests for disaster relief resources and tracks their statuses.
- **Donor:** Registers resource donations and tracks their fulfillment.
- **Volunteer:** Registers skills, updates availability, and reports task progress.
- **Organization Coordinator:** Verifies requests, manages warehouse inventory, allocates resources, and assigns tasks.
- **System Administrator:** Manages user roles and verifies newly registered organizations.

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

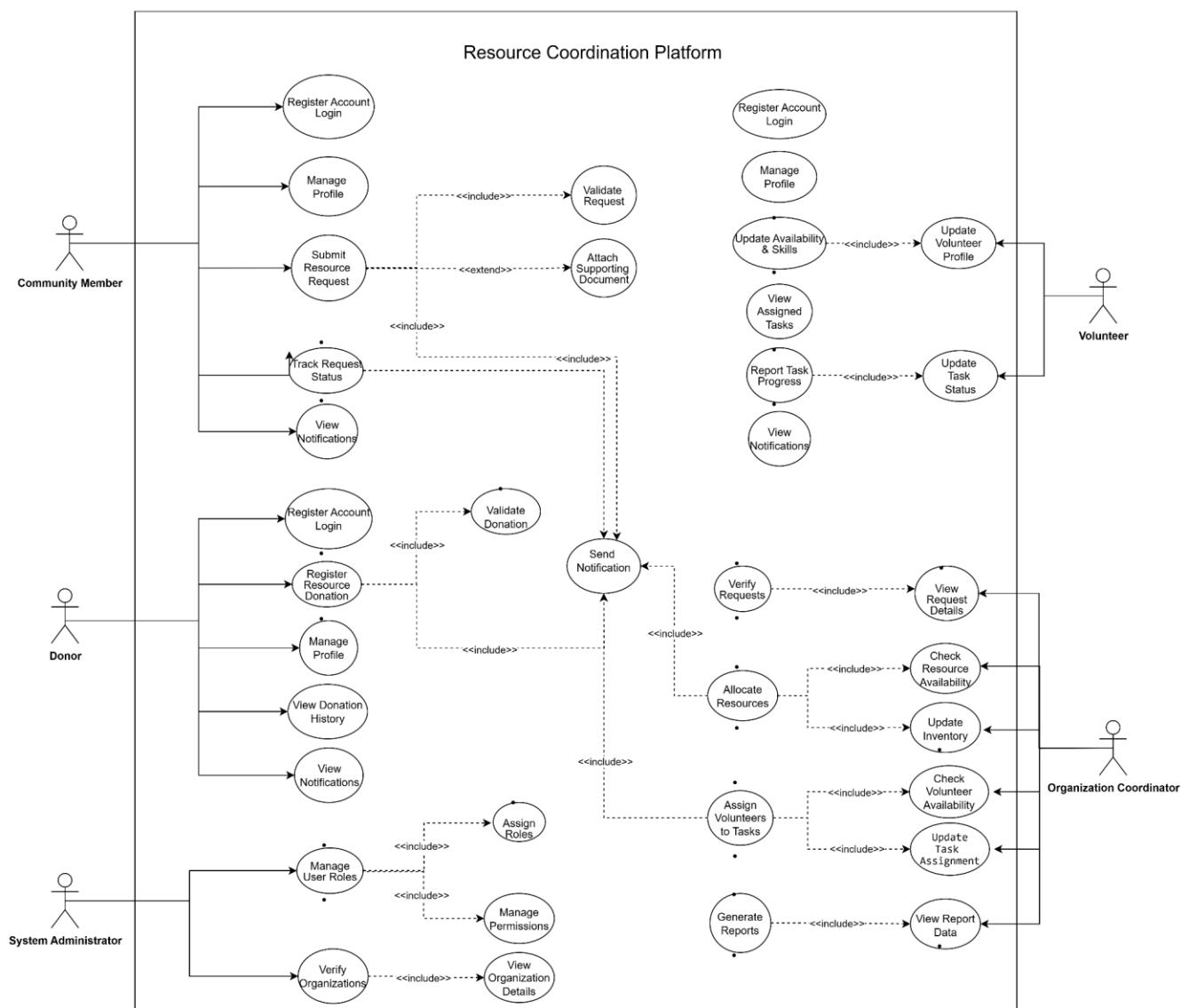


Figure 3 Use Case Diagram

Diagram Description: Figure 4 illustrates the system boundaries and interactions between the primary actors and core functionalities. It highlights major use cases including User Registration, Submit Resource Request, Allocate Resources, Assign Volunteer, and Generate Reports, demonstrating the isolation of privileges across the platform.

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

4.1 Use-Case Realizations

The following scenarios illustrate how the software executes its most architecturally significant functionalities, detailing how various design model elements contribute to the system.

Use Case 1: Submit Resource Request

Use case name	Submit Resource Request
Actor	Community Member, Organization Coordinator
Description	Allows authorized users to submit a distress or relief request specifying the needed resources and emergency location.
preconditions	The user must be authenticated via the IAM Framework and have an active account status.
Main flow	1. User navigates to the Request Creation interface. 2. User enters resource category, required quantity, priority level, location, and required date. 3. User submits the form. 4. Request Service validates the input data. 5. System saves the request to the PostgreSQL database with a "PENDING" status. 6. System publishes an event to Apache Kafka to trigger the Notification Service.
Successful end/post condition	Request is successfully stored with a Pending status, and Coordinators receive a notification alert.
Fail end/post condition	Request is rejected due to invalid data or network failure, returning an HTTP 400 error and displaying an informative message to the user.
Extensions	If the mobile application is offline, the request is cached locally and synchronized automatically when the network connection is restored.

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

Use Case 2: Track Request Status

Use case name	Track Request Status
Actor	Community Member
Description	Allows a community member to view the current status and progress of submitted resource requests.
preconditions	The user must be authenticated and must have submitted at least one resource request.
Main flow	1. User opens the My Requests interface. 2. The mobile application sends a request to the Request Service through Kong API Gateway. 3. Request Service verifies the user's identity and ownership of the request. 4. Request Service retrieves the request details and current status from PostgreSQL. 5. The system displays the request status and relevant fulfillment information.
Successful end/post condition	The user can view the latest status of the submitted request
Fail end/post condition	The request details cannot be retrieved due to authorization, database, or network failure.
Extensions	If the request status changes, the Notification Service may send a notification to the requester through Kafka-based event processing.

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

Use Case 3: Register Donation Resource

Use case name	Register Donation Resource
Actor	Donor
Description	Allows a donor to register an offer to donate resources to an organization or to fulfill an approved resource request.
Preconditions	The donor must be authenticated and have an active Donor account.
Main flow	1. Donor selects a resource donation option. 2. Donor specifies resource category, quantity, target organization or request, and delivery information. 3. Donor submits the donation offer. 4. Resource Service validates the donation details. 5. The donation is stored in PostgreSQL with a 'PENDING' status. 6. Resource Service publishes a donation-created event to Kafka. 7. Notification Service notifies the relevant Organization Coordinator.
Successful end/post condition	The donation offer is recorded successfully and the organization is notified
Fail end/post condition	The donation is rejected because of invalid information, unauthorized access, or a system failure.
Extensions	If the donation is made without a specific help request, it is associated directly with the selected organization and remains Pending until reviewed.

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

Use Case 4: View Donation History

Use case name	View Donation History
Actor	Donor
Description	Allows donors to view their previous and current donation records and their fulfillment status
preconditions	The donor must be authenticated.
Main flow	1. Donor opens the Donation History interface. 2. The application sends a request to the Resource Service through Kong. 3. Resource Service verifies the donor identity. 4. Resource Service retrieves the donor's donation records from PostgreSQL. 5. The system displays donation details and statuses..
Successful end/post condition	The donor can view their donation history and current donation statuses.
Fail end/post condition	Donation records cannot be retrieved because of authorization, database, or network failure.
Extensions	Large donation histories are retrieved using pagination

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

Use Case 5: Verify Donation

Use case name	Verify Donation
Actor	Organization Coordinator
Description	Allows an Organization Coordinator to review and approve or reject a donor's resource donation offer.
preconditions	The coordinator must be authenticated and authorized to manage the organization's donations. A donation must have a Pending status.
Main flow	1. Coordinator opens the pending donation list. 2. Coordinator selects a donation and views its details. 3. Resource Service verifies the coordinator's organization permissions. 4. Coordinator approves or rejects the donation. 5. Resource Service updates the donation status in PostgreSQL. 6. Resource Service publishes a donation-verification event to Kafka. 7. Notification Service notifies the donor of the decision.
Successful end/post condition	The donation status is updated to Approved or Rejected and the donor is notified.
Fail end/post condition	The donation cannot be verified due to authorization, validation, or system failure.
Extensions	An approved donation proceeds to the resource receipt and inventory update process.

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

Use Case 6: Verify Request

Use case name	Verify Request
Actor	Organization Coordinator
Description	Allows an Organization Coordinator to review submitted resource requests and approve or reject them based on the provided information.
preconditions	The coordinator must be authenticated and authorized to review requests for the organization. The request must have a Pending status.
Main flow	1. Coordinator views the list of pending requests. 2. Coordinator selects a request. 3. Request Service retrieves the request details. 4. Coordinator approves or rejects the request. 5. Request Service updates the request status. 6. A status-change event is published to Kafka. 7. Notification Service notifies the requester of the decision.
Successful end/post condition	The request is updated to Approved or Rejected and the requester is notified.
Fail end/post condition	The request cannot be verified because of invalid data, authorization failure, or system failure.
Extensions	An approved request becomes eligible for resource allocation and may be displayed to donors.

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

Use Case 7: Allocate Resources

Use case name	Allocate Resources
Actor	Organization Coordinator
Description	Allows an Organization Coordinator to allocate available inventory resources to fulfill an approved emergency resource request.
preconditions	The request must be Approved and sufficient inventory must be available. The coordinator must have appropriate permissions.
Main flow	1. Coordinator selects an approved request. 2. Coordinator initiates the resource allocation process. 3. Resource Service checks inventory availability. 4. System verifies that sufficient stock exists. 5. Required quantities are reserved and inventory is updated. 6. A ResourceAllocation record is created. 7. Request fulfillment status is updated. 8. A fulfillment event is published through Kafka. 9. Notification Service notifies the requester.
Successful end/post condition	Resources are allocated successfully, inventory is updated, and the request status becomes Partially Fulfilled or Fulfilled.
Fail end/post condition	Allocation is aborted if sufficient resources are unavailable or a transaction fails. Inventory remains unchanged.
Extensions	If only part of the requested quantity is available, a partial allocation is recorded and the remaining quantity stays pending.

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

Use Case 8: Update Inventory

Use case name	Update Inventory
Actor	Organization Coordinator
Description	Allows an authorized coordinator to update organizational inventory when resources are received, allocated, released, or otherwise modified.
preconditions	The coordinator must be authenticated and authorized to manage organizational inventory.
Main flow	1. Coordinator opens the inventory management interface. 2. Coordinator selects an inventory item. 3. Coordinator enters the required quantity or inventory change. 4. Inventory Service validates the operation. 5. The inventory quantity is updated in PostgreSQL. 6. An inventory transaction is recorded. 7. Relevant events are published through Kafka when necessary.
Successful end/post condition	Inventory records accurately reflect the latest available and reserved quantities.
Fail end/post condition	The update is rejected if the quantity is invalid, the item does not exist, or the user is unauthorized.
Extensions	Inventory updates may be automatically triggered when an approved donation is physically received or resources are allocated.

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

Use Case 9: Update Availability and Skills

Use case name	Update Availability and Skills
Actor	Volunteer
Description	Allows volunteers to maintain their skills, experience, availability, and other information used for task assignment.
preconditions	The volunteer must be authenticated and have an active volunteer profile.
Main flow	1. Volunteer opens the profile management interface. 2. Volunteer enters or modifies skills, experience, availability, and preferred working areas. 3. Volunteer submits the changes. 4. Volunteer Service validates the information. 5. Volunteer profile is updated in PostgreSQL. 6. Updated availability and skills become available for task matching.
Successful end/post condition	The volunteer profile contains the latest skills and availability information.
Fail end/post condition	The update is rejected because of invalid information, authorization failure, or system error.
Extensions	Changes to volunteer availability may be considered when coordinators search for suitable volunteers.

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

Use Case 10: Assign Volunteers to Tasks

Use case name	Assign Volunteers to Tasks
Actor	Organization Coordinator
Description	Allows a coordinator to assign suitable volunteers to disaster-response tasks based on their skills and availability.
preconditions	The coordinator must be authorized, a task must exist, and suitable verified volunteers must be available.
Main flow	1. Coordinator creates or selects a task. 2. Task Service retrieves suitable volunteers from the Volunteer Service. 3. Volunteer skills and availability are checked. 4. Coordinator selects a suitable volunteer. 5. Task Service creates the assignment record. 6. The assignment is stored in PostgreSQL. 7. An assignment event is published to Kafka. 8. Notification Service notifies the selected volunteer.
Successful end/post condition	The volunteer is successfully assigned to the task and receives a notification.
Fail end/post condition	Assignment fails if no suitable volunteer is available or the coordinator does not have sufficient permissions.
Extensions	A coordinator may assign multiple volunteers to a single task when required.

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

Use Case 11: Accept or Reject Task

Use case name	Accept or Reject Task
Actor	Volunteer
Description	Allows a volunteer to accept or reject a task assigned by an Organization Coordinator.
preconditions	The volunteer must be authenticated and have a pending task assignment.
Main flow	1. Volunteer receives a task assignment notification. 2. Volunteer views the task details, location, priority, and deadline. 3. Volunteer chooses to accept or reject the task. 4. Task Service validates the volunteer's assignment. 5. Task Service updates the assignment status. 6. A task-status event is published to Kafka. 7. Notification Service informs the Organization Coordinator.
Successful end/post condition	The assignment is updated to Accepted or Rejected and the coordinator is notified.
Fail end/post condition	The decision cannot be recorded due to an invalid assignment, authorization failure, or system error.
Extensions	If the volunteer rejects the task, the coordinator may assign another available volunteer.

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

Use Case 12: Report Task Progress

Use case name	Report Task Progress
Actor	Volunteer
Description	Allows an assigned volunteer to report progress and provide updates while completing a disaster-response task.
preconditions	The volunteer must be authenticated and assigned to an active task.
Main flow	1. Volunteer opens the assigned task. 2. Volunteer enters progress information and optional notes. 3. Volunteer submits the update. 4. Task Service verifies that the volunteer is the authorized assignee. 5. Progress information and timestamp are stored in PostgreSQL. 6. A progress event is published to Kafka. 7. The coordinator is notified or the dashboard is updated.
Successful end/post condition	The task contains the latest progress information and the coordinator can monitor the task.
Fail end/post condition	The progress update is rejected if the volunteer is unauthorized or the task is no longer active.
Extensions	Multiple progress updates may be recorded throughout the lifetime of a task.

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

Use Case 13: Notification Delivery

Use case name	Notification Delivery
Actor	Notification Service
Description	Delivers system notifications to users when important events occur, such as request verification, donation approval, resource allocation, task assignment, or task completion.
preconditions	A relevant domain event must be published to Apache Kafka and the target user must have a valid account.
Main flow	1. A backend service publishes a domain event to Kafka. 2. Notification Service consumes the event. 3. Notification Service identifies the affected user or users. 4. User notification preferences are retrieved. 5. The notification is generated. 6. The notification is delivered through the appropriate channel. 7. Delivery status is recorded in PostgreSQL.
Successful end/post condition	The notification is successfully delivered or its delivery status is recorded for retry.
Fail end/post condition	If delivery fails, the system records the failure and may retry the notification.
Extensions	Notifications may be delivered through in-app, push, or email channels depending on user preferences.

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

Use Case 14: Verify Organization

Use case name	Verify Organization
Actor	Organization Coordinator, System Administrator
Description	Allows an organization to register on the platform and enables a System Administrator to verify and approve the organization.
preconditions	The organization coordinator must provide the required organization information and registration documents.
Main flow	1. Organization Coordinator submits organization details and supporting documents. 2. Organization Service validates the submitted information. 3. Organization Service checks organization uniqueness. 4. A Pending organization record is created. 5. System Administrator reviews the organization details and documents. 6. Administrator approves or rejects the registration. 7. Organization Service updates the organization status. 8. Notification Service informs the organization coordinator
Successful end/post condition	The organization is successfully verified and its status is updated to Approved, or it is rejected with an appropriate status.
Fail end/post condition	Registration fails because of invalid information, duplicate organization details, missing documents, or system failure.
Extensions	The administrator may request additional information or documents before making the final decision.

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

Use Case 15: Manage User Roles and Permissions

Use case name	Manage User Roles
Actor	System Administrator
Description	Allows the System Administrator to assign, modify, or revoke roles and permissions for platform users.
preconditions	The administrator must be authenticated and have system-level administrative privileges.
Main flow	1. Administrator opens the user management interface. 2. Administrator searches for a user. 3. System retrieves the user's current roles and permissions. 4. Administrator assigns, changes, or removes a role. 5. IAM/User Service validates the operation. 6. The updated role and permission information is stored. 7. The system applies the updated RBAC configuration.
Successful end/post condition	The user's roles and permissions are successfully updated.
Fail end/post condition	The operation is rejected if the administrator is unauthorized or the requested role configuration is invalid.
Extensions	Role changes may affect the user's access to web and mobile application functionality immediately after the next authorization check.

5. Logical View

This section describes the architecturally significant parts of the design model, illustrating the decomposition of the platform into distinct microservices and their internal class structures.

5.1 Overview

The logical architecture of the Resource Coordination Platform follows a Microservice Architecture pattern. The system is decoupled into independent functional domains (packages), which communicate asynchronously via Apache Kafka and synchronously via REST APIs.

Core Subsystems (Packages):

- **API Gateway Package:** Routes incoming client requests the appropriate backend microservice and enforces JWT token validation.
- **User & IAM Service Package:** Handles authentication, token generation, profile management, and RBAC implementation.
- **Request Service Package:** Manages the entire life cycle of emergency resource requests (creation, approval, cancellation).
- **Resource Service Package:** Acts as the central ledger for donations, warehouse inventory tracking, and

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

physical resource allocations.

- **Task & Volunteer Service Packages:** Coordinates human resources, tracks volunteer skills, availability, and manages field assignments.
- **Notification Service Package:** A decoupled event consumer that dispatches email, push, and in-app alerts based on subscribed Kafka topics.

5.2 Architecturally Significant Design Packages

This subsection decomposes the core microservices into their architecturally significant classes, outlining their responsibilities, attributes, and relationships.

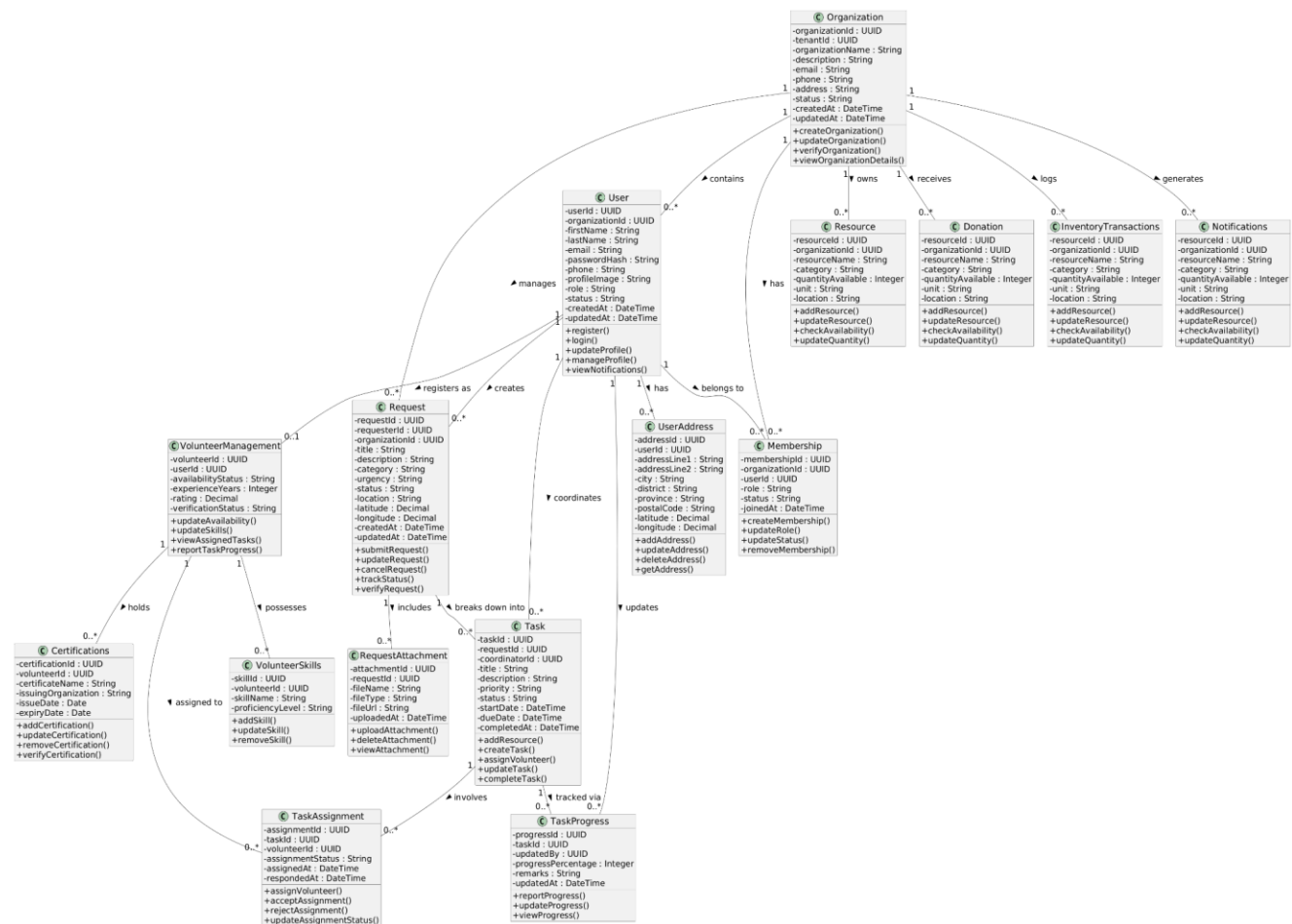


Figure 5 Class Diagram

Diagram Description: Figure 2 displays the logical schema of the core platform entities. It shows the aggregation between the Organization and User classes, the one-to-many relationship between ResourceRequest and ResourceAllocation, and the dependencies between the Task controller and Volunteer entities across service boundaries.

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

Package 1: RequestService This package handles the business logic for creating, reviewing, and tracking disaster requests.

- **Class: ResourceRequest**
 - *Brief Description:* Represents a community member's plea for resources during an emergency.
 - *Attributes:* requestId (UUID), requesterId (UUID), resourceCategory (String), quantity (Integer), priorityLevel (Enum), status (Enum), location (String).
 - *Operations:* createRequest(), updateStatus(), cancelRequest().
- **Class: RequestController**
 - *Brief Description:* The REST API endpoint handler for request-related HTTP methods.
 - *Operations:* handlePostRequest(), handleGetRequestsByOrg(), validateRequestData().

Package 2: ResourceService This package serves as the inventory ledger, ensuring data consistency and preventing race conditions during high-concurrency disaster allocations.

- **Class: InventoryItem**
 - *Brief Description:* Represents a physical resource stored in an organizational warehouse.
 - *Attributes:* itemId (UUID), category (String), availableQuantity (Integer), reservedQuantity (Integer), expiryDate (Date).
 - *Operations:* reserveStock(), releaseStock(), updateQuantity().
- **Class: ResourceAllocation**
 - *Brief Description:* Maps specific inventory items to approved resource requests, acting as a historical ledger.
 - *Attributes:* allocationId (UUID), requestId (UUID), allocatedQuantity (Integer), allocationDate (Date).
 - *Operations:* executeAllocation(), generateAllocationReport().

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

6. Process View

6.1 Activity Diagrams

Activity diagrams visually illustrate the overall flow of activities, processes, and interactions across The Resource Coordination Platform. They show how users (Community members, donors, volunteers, Coordinators and Admins) and services (User, Request, Organization, Task, Volunteer, Resource, Notification) collaborate over time to complete business tasks

6.1.1 User Registration and Login

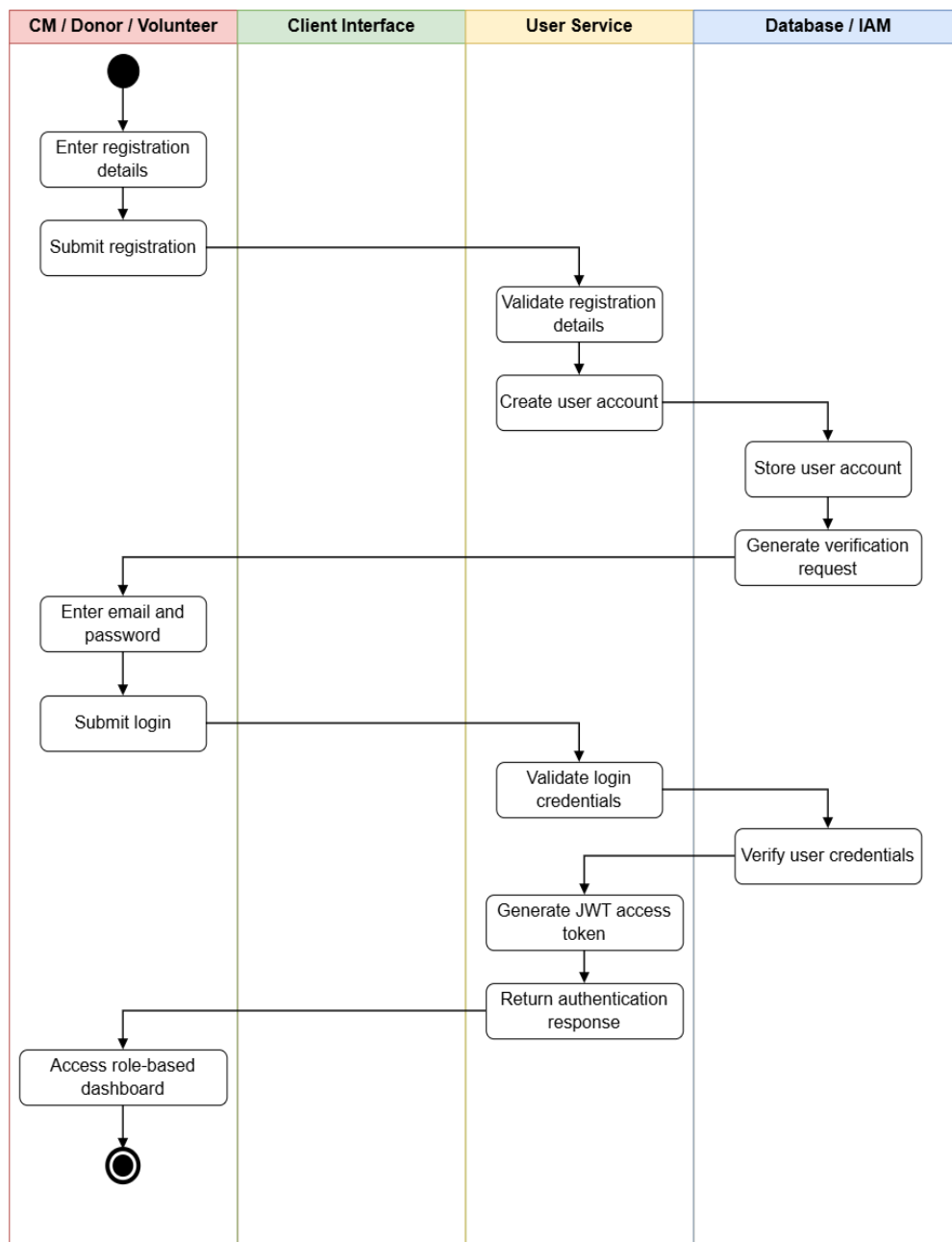


Figure 6 User Registration and Login

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

6.1.2 Help Request Submission

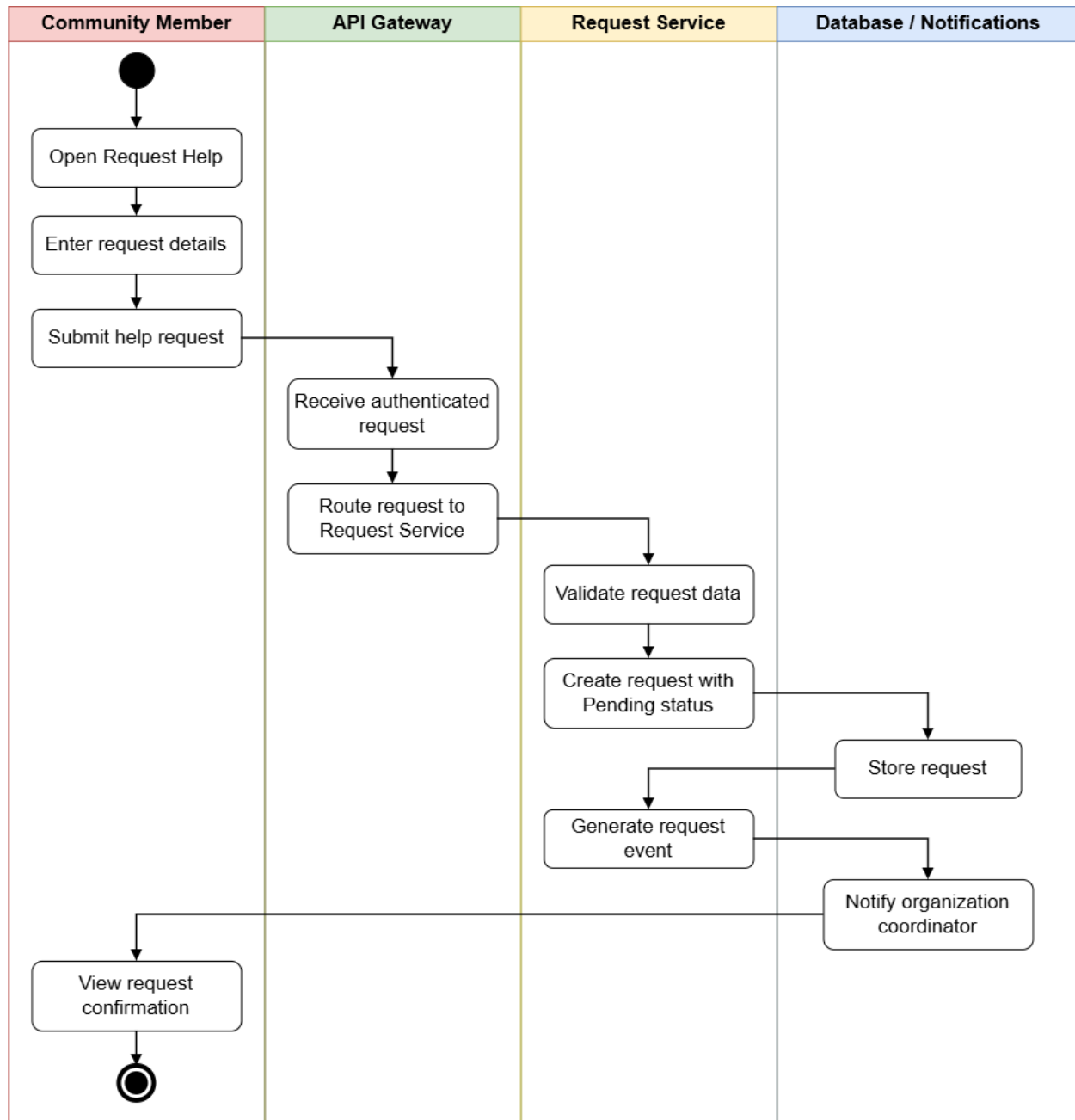


Figure 7 Help Request Submission

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

6.1.3 Organization Coordinator Reviews a Help Request

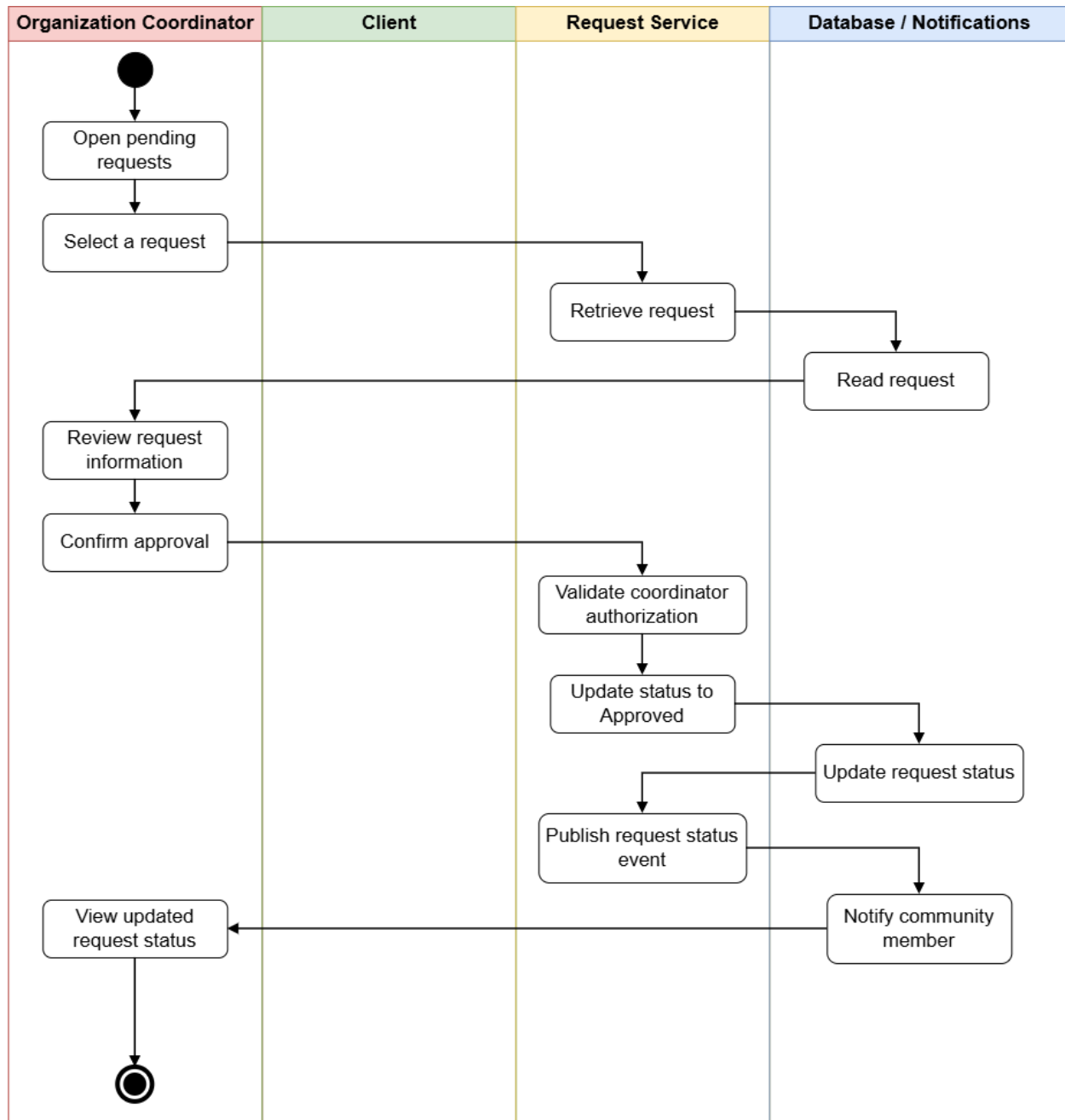


Figure 8 Organization Coordinator Reviews a Help Request

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

6.1.4 Donor views Approved Needs and Offers Donation

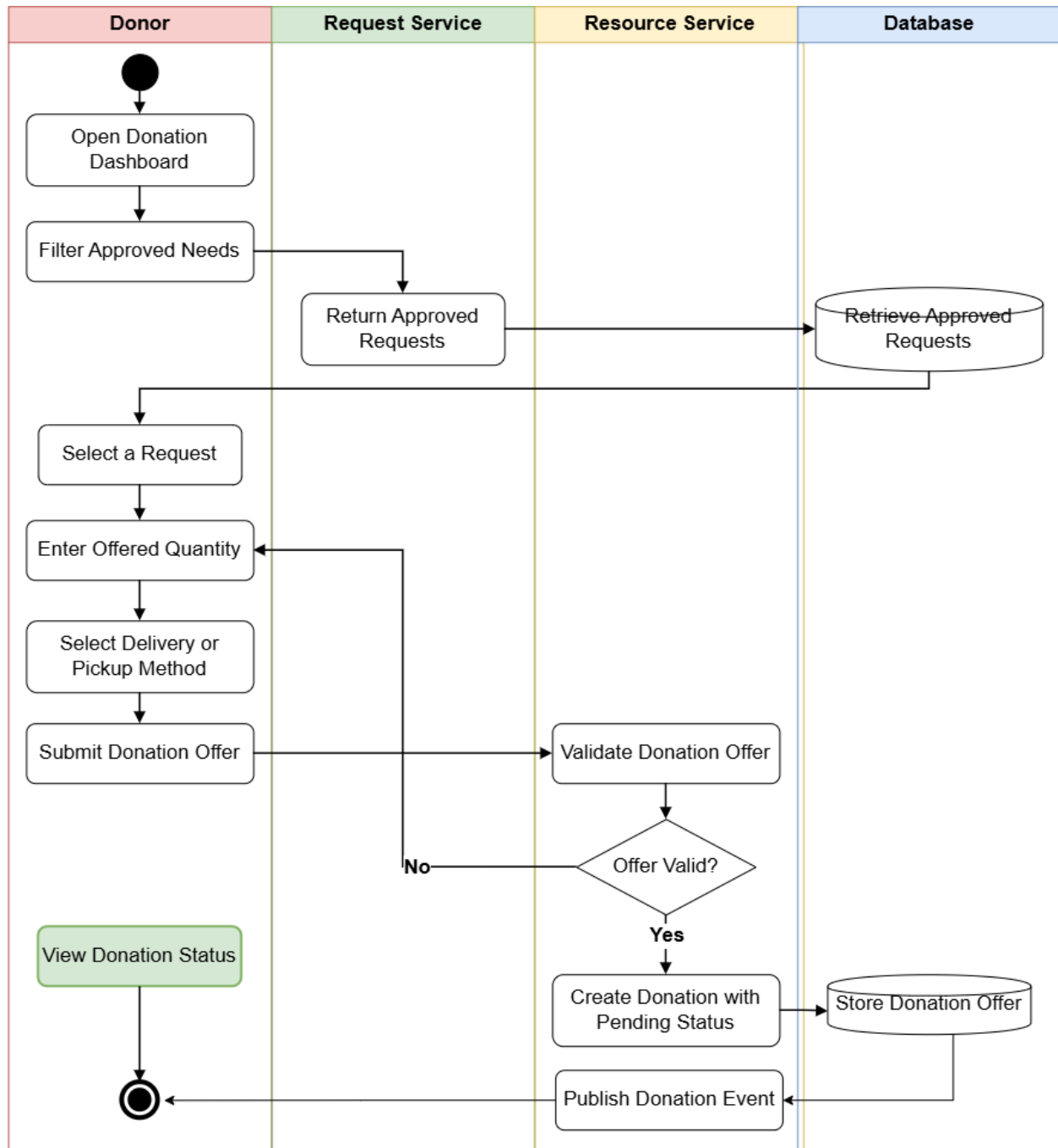


Figure 9 Donor views Approved Needs and Offers Donation

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

6.1.5 Organization Coordinator Verifies a Donation

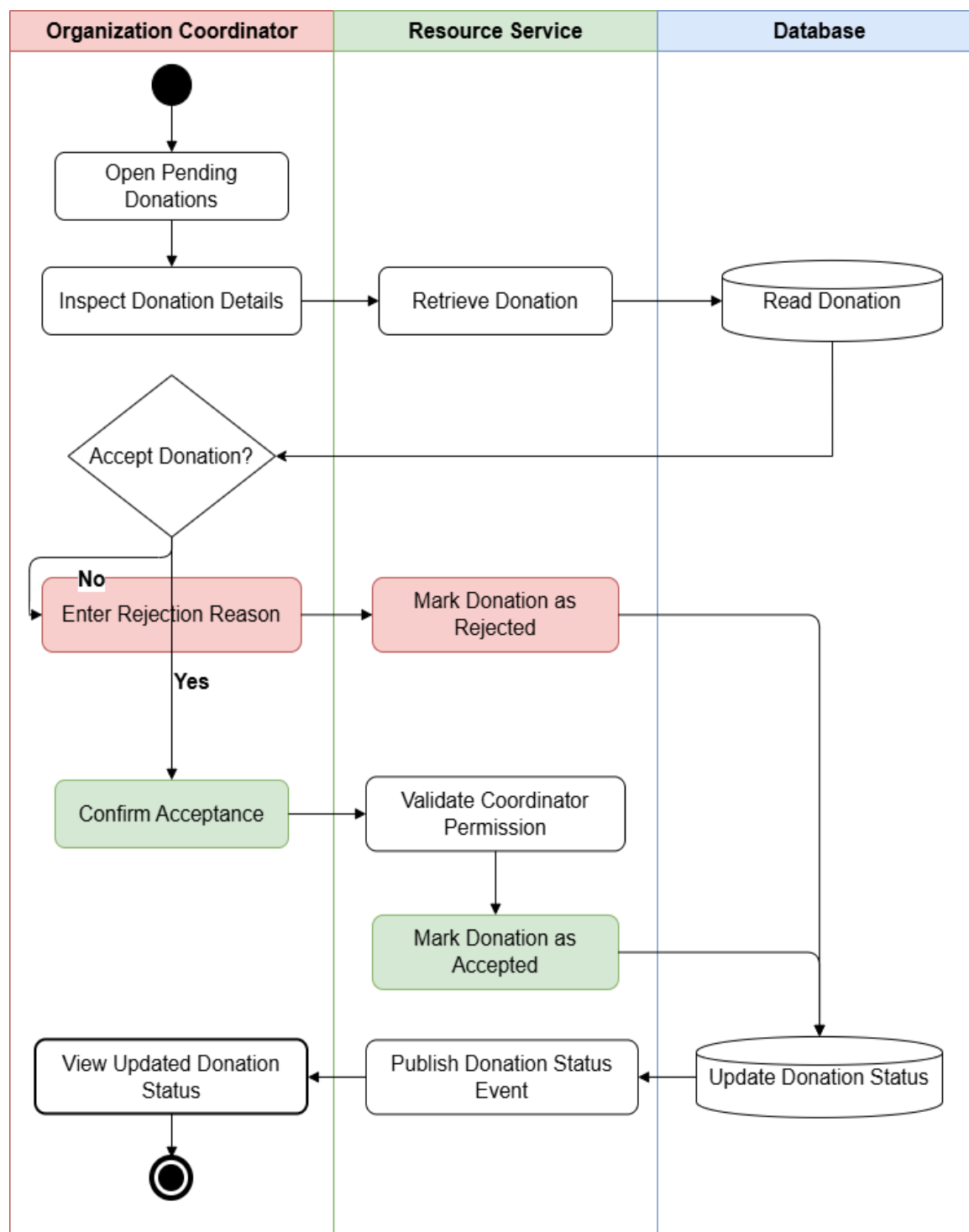


Figure 10 Organization Coordinator Verifies Donation

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

6.1.6 Donation Receipt and Inventory Update

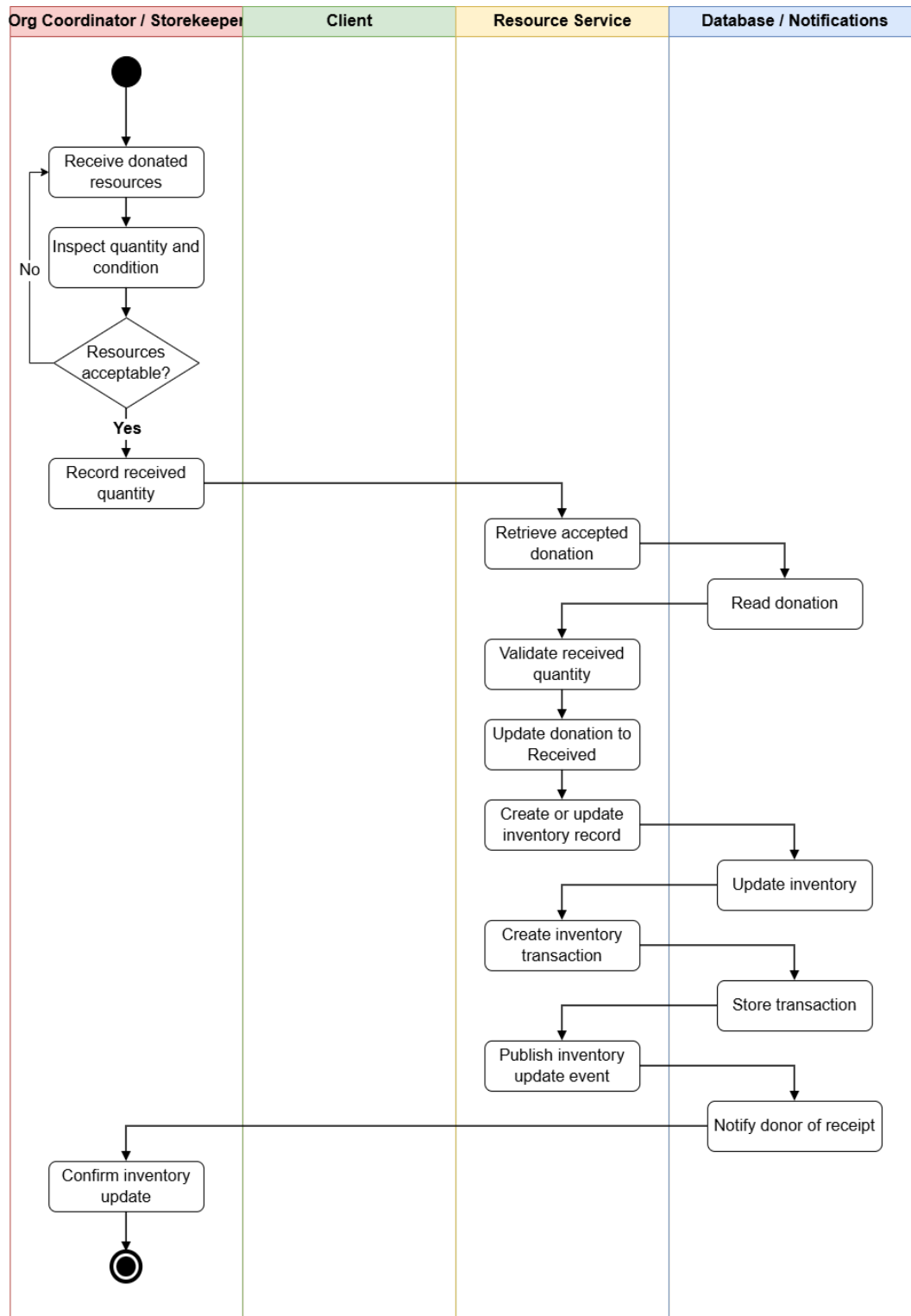


Figure 11 Donation Receipt and Inventory Update

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

6.1.7 Direct Donation Without a Specific Help Request

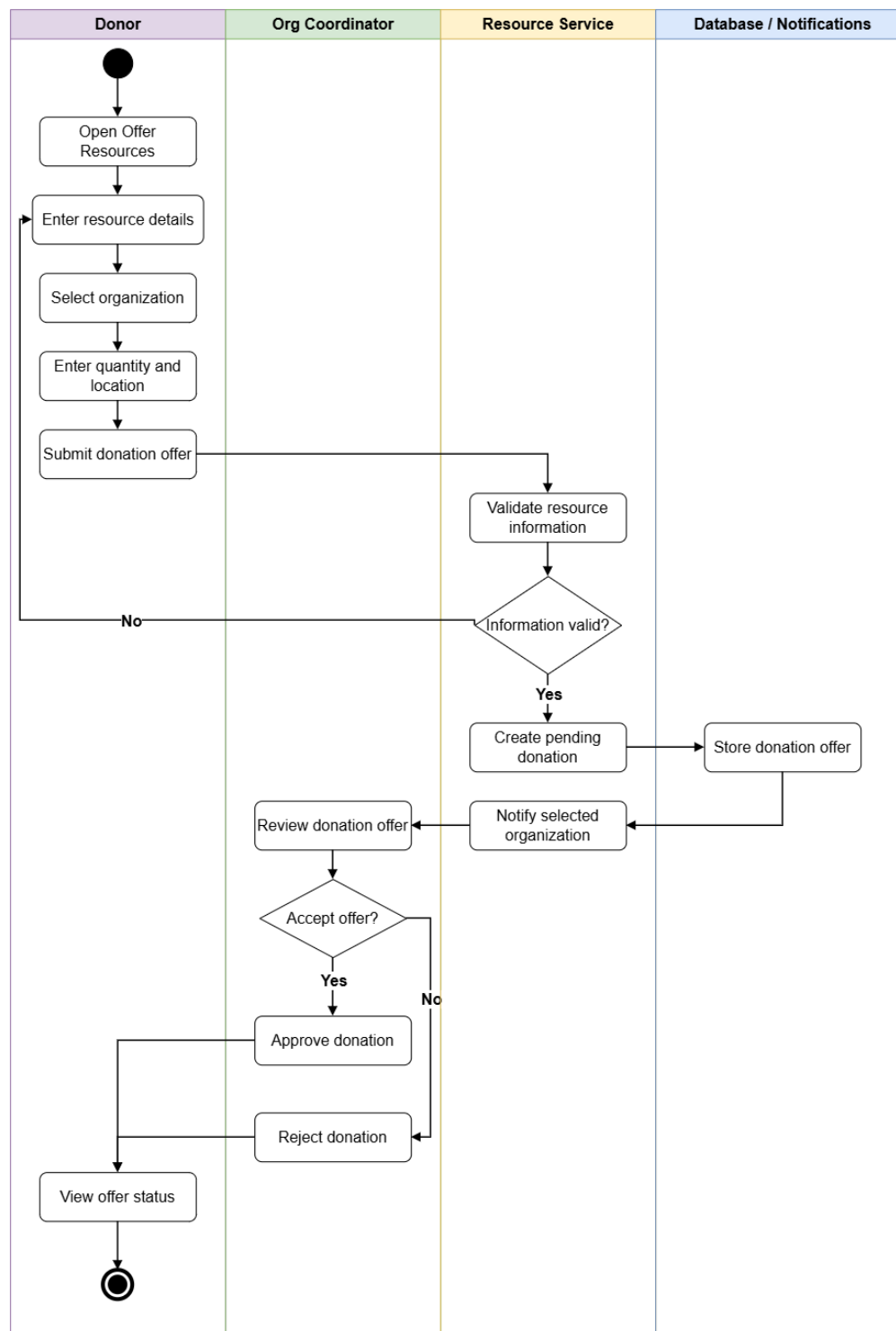


Figure 12 Direct Donation Without a Specific Help Request

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

6.1.8 Resource Allocation to an Approved Request

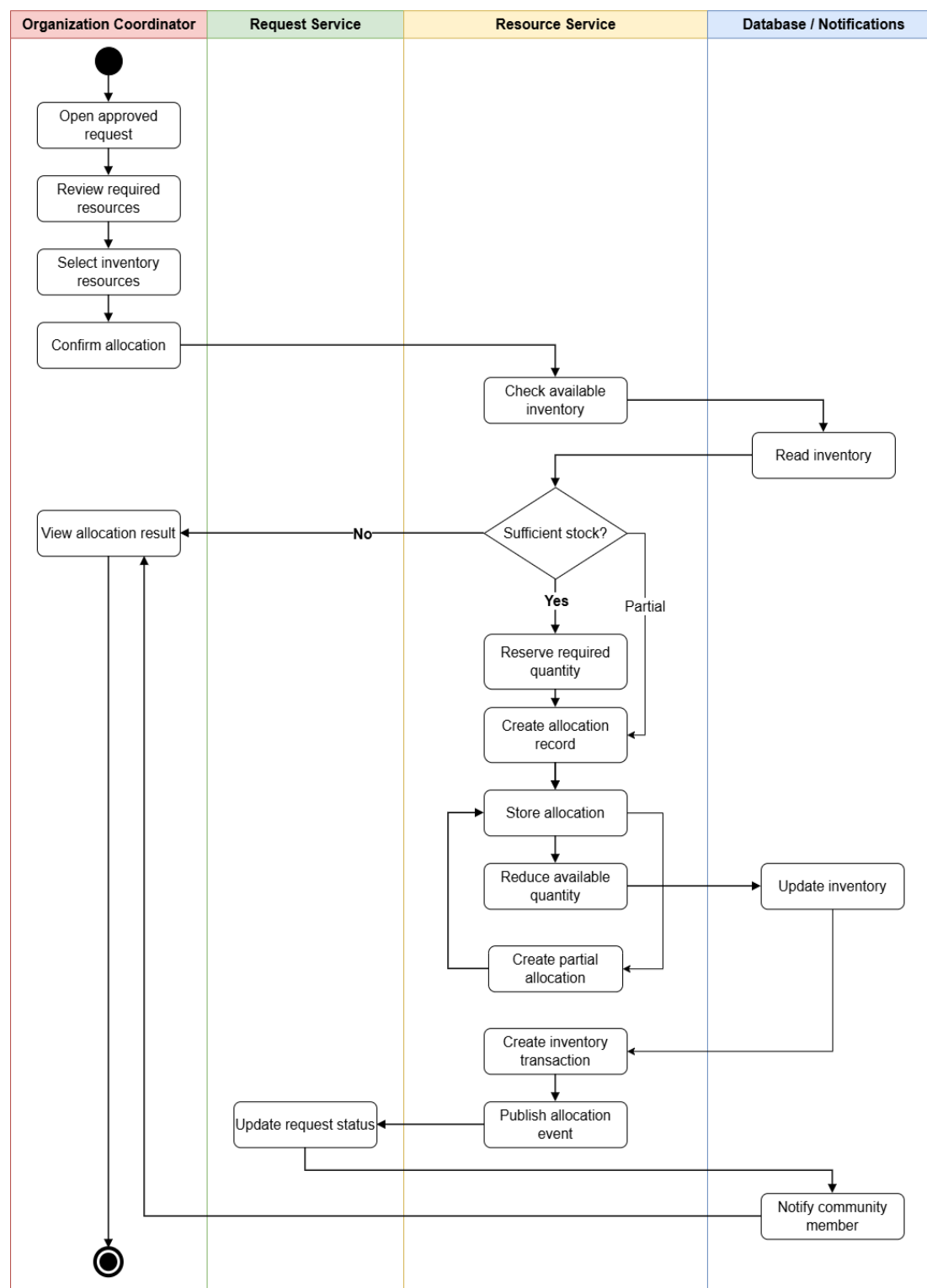


Figure 13 Resource Allocation to an Approved Request

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

6.1.9 Volunteer Registration and Verification

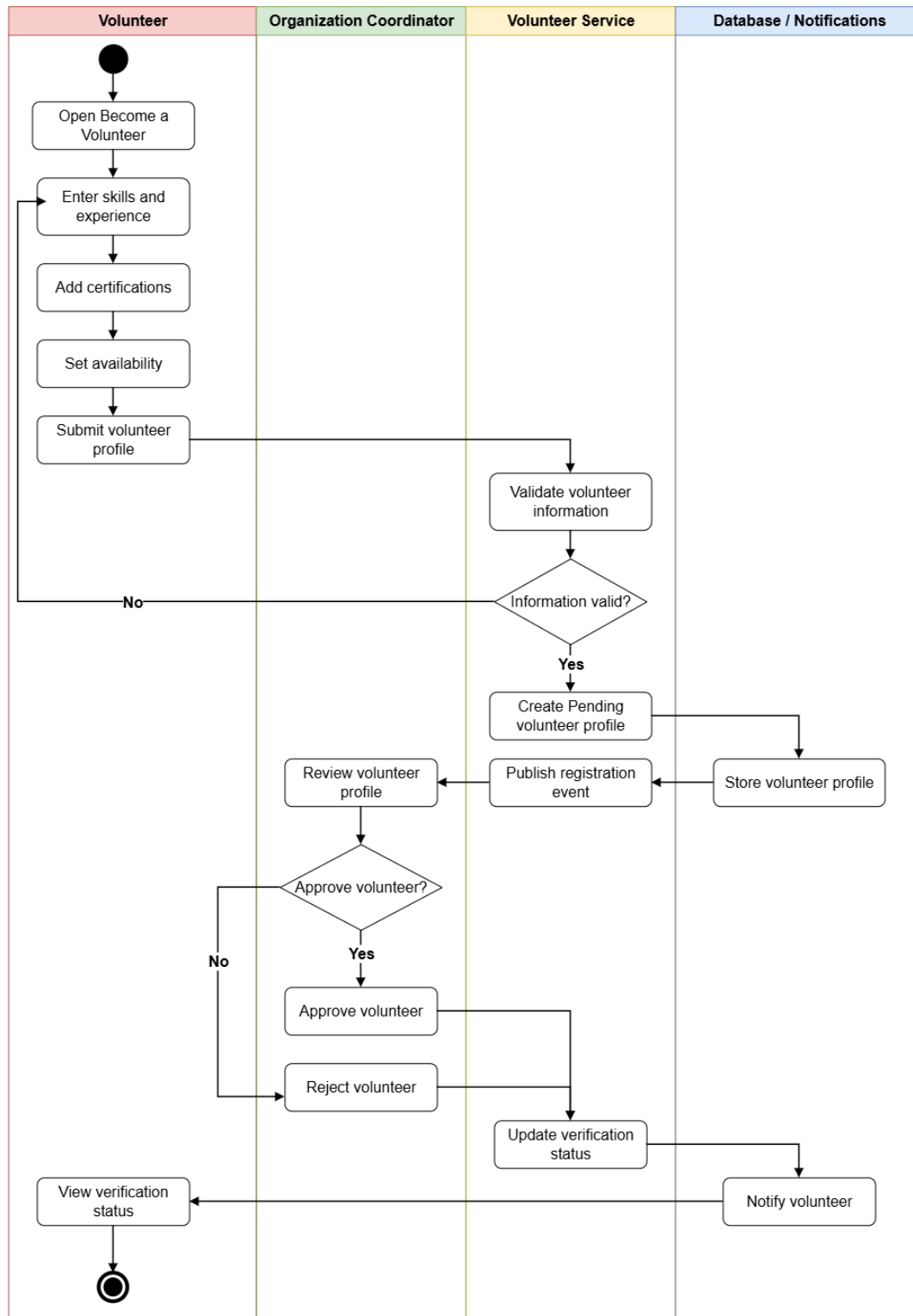


Figure 14 Volunteer Registration and Verification

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

6.1.10 Task Creation and Volunteer Assignment

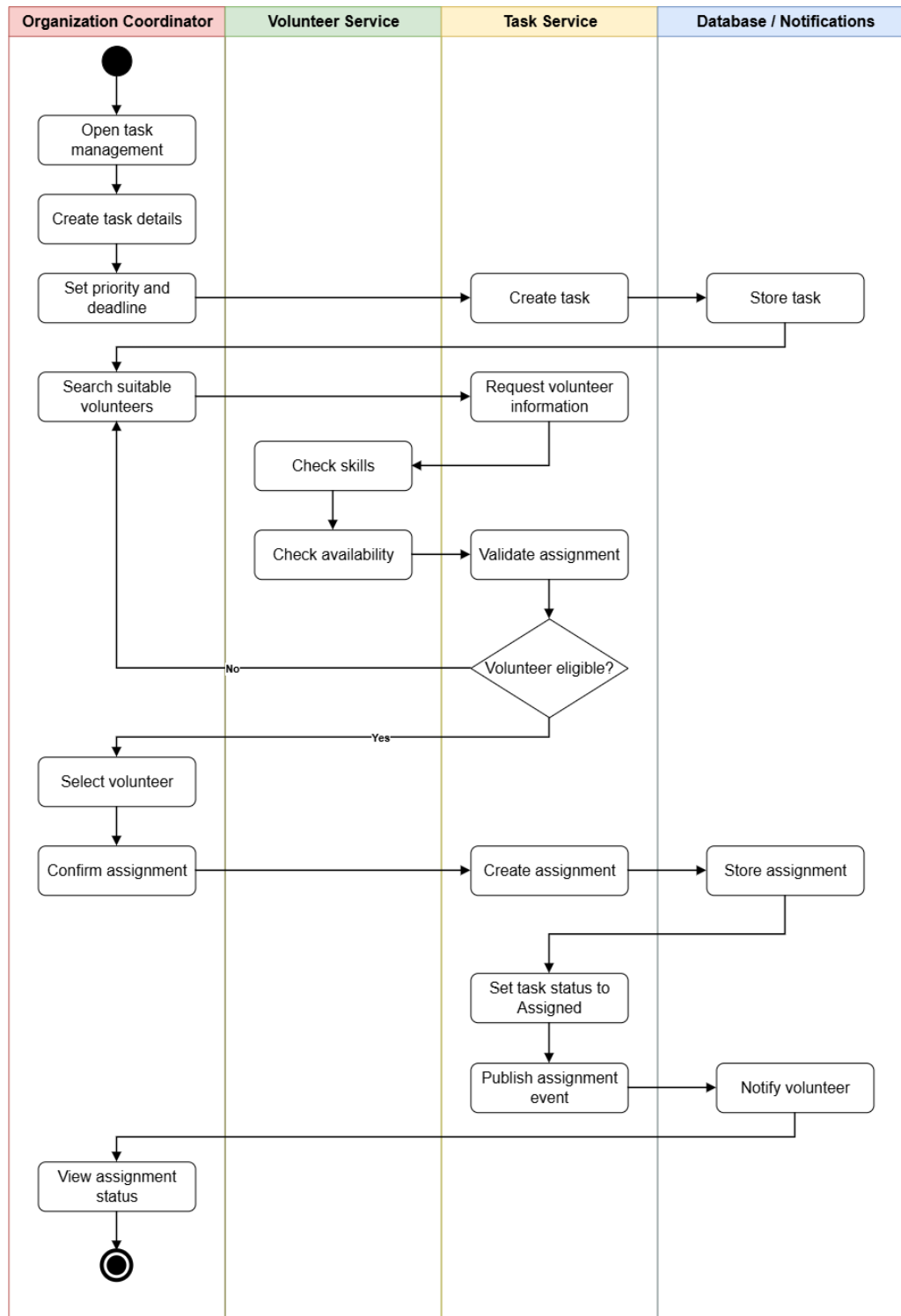


Figure 15 Task Creation and Volunteer Assignment

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

6.1.11 Volunteer Accepts or Rejects a Task

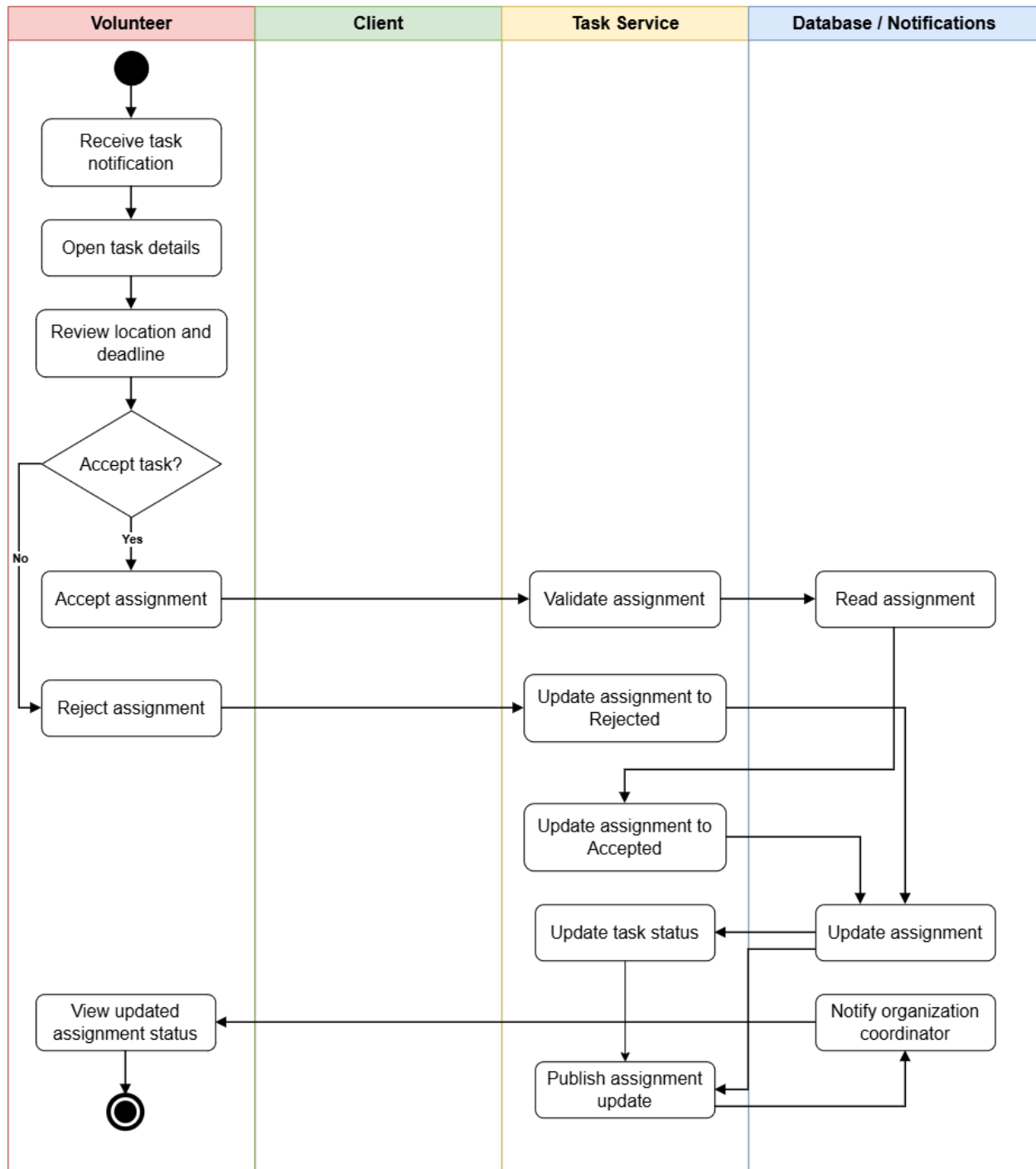


Figure 16 Volunteer Accepts or Rejects a Task

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

6.1.12 Task Completion

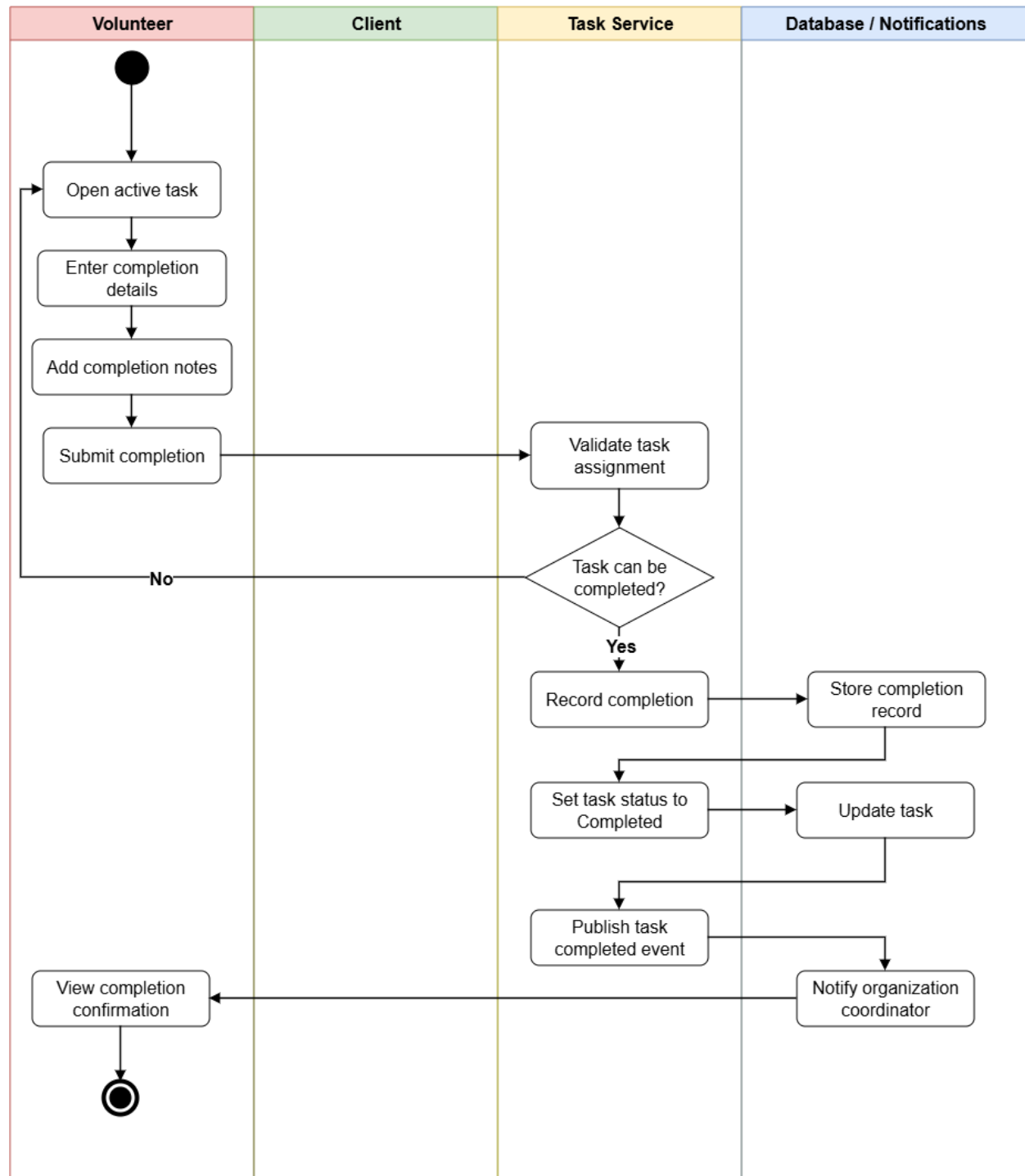


Figure 17 Task Completion

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

6.1.13 Notification Delivery

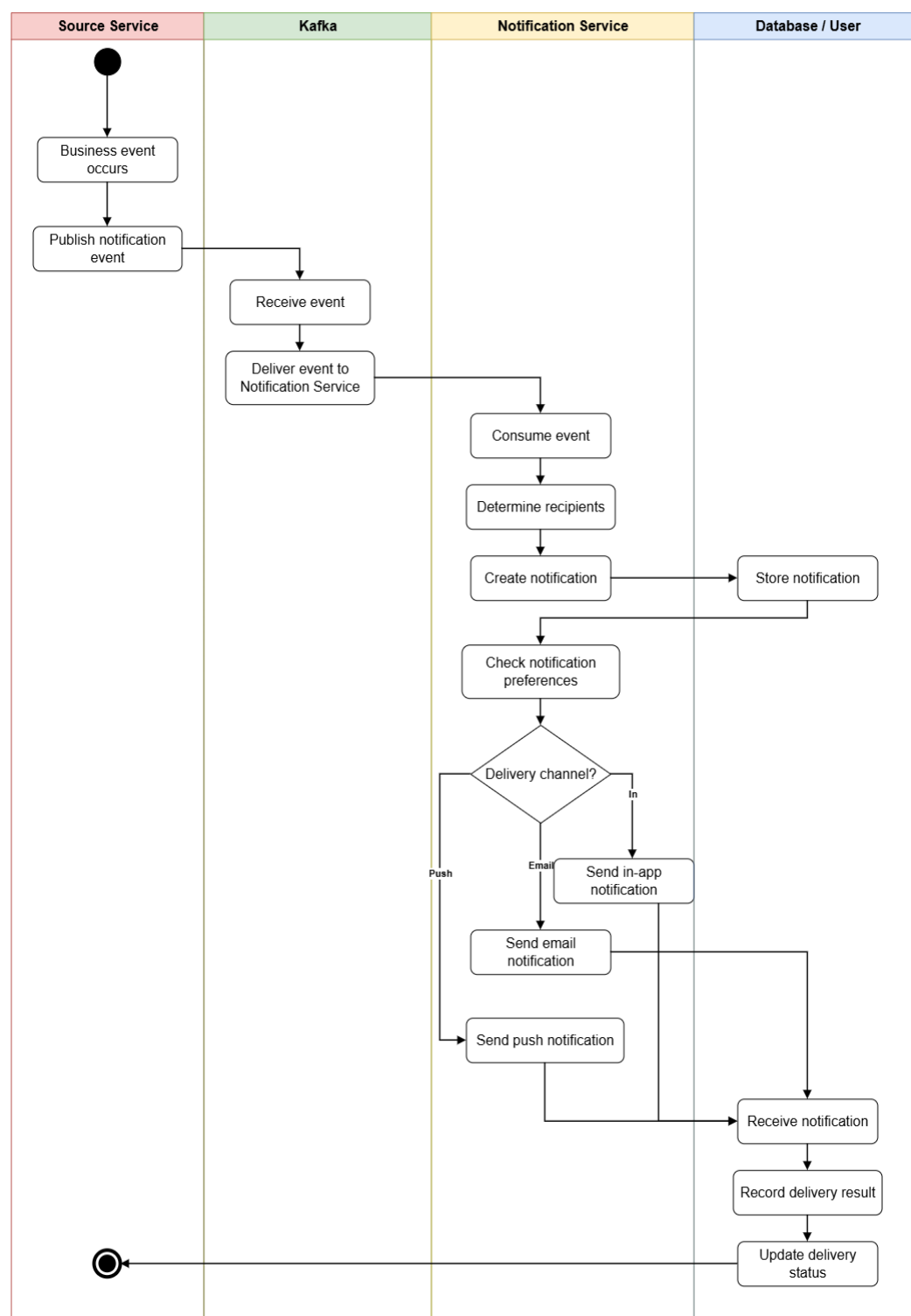


Figure 18 Notification Delivery

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

6.1.14 Organization Registration and Approval

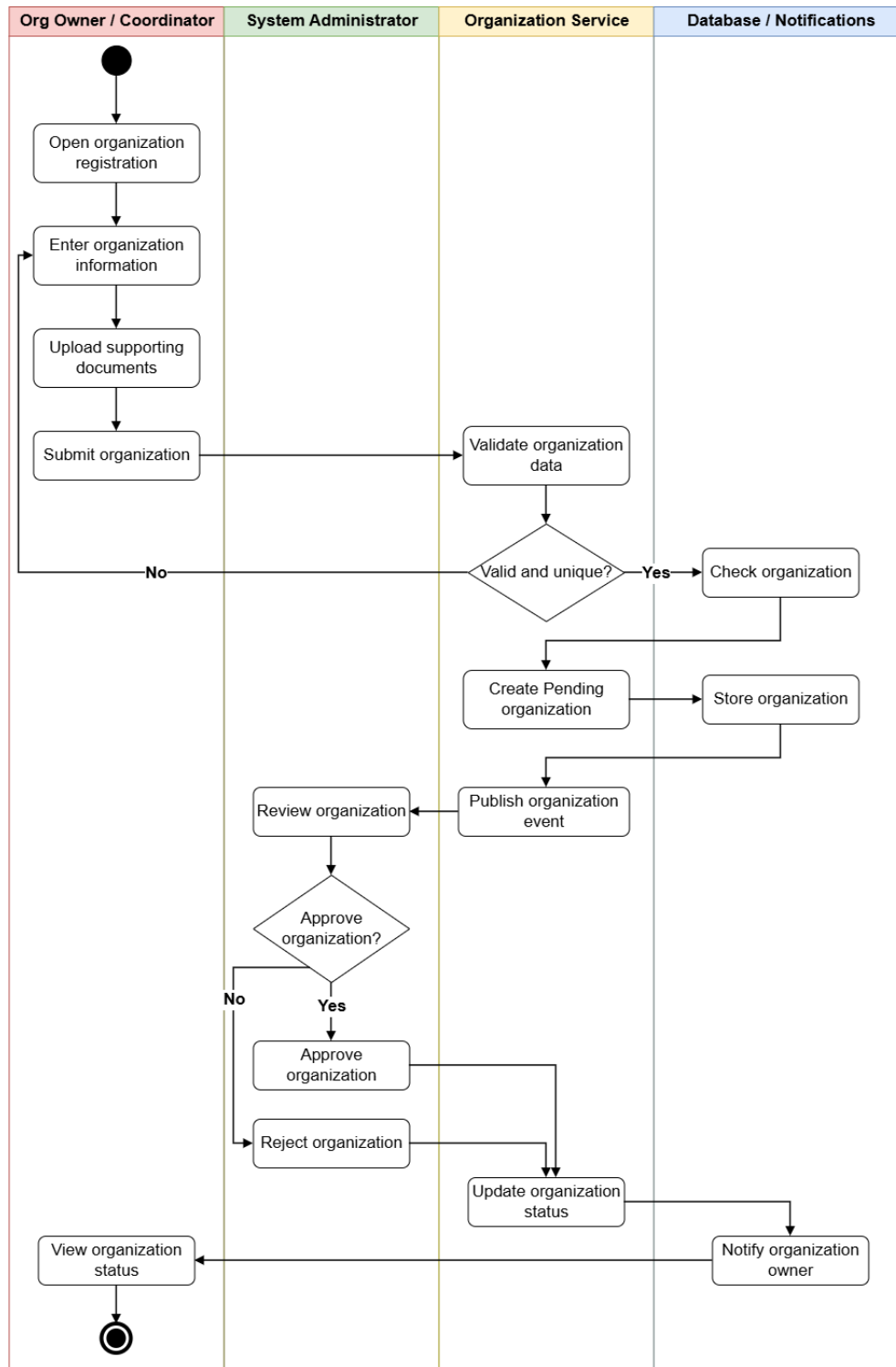


Figure 19 Organization Registration and Approval

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

6.2 Sequence Diagrams

This section provides the Sequence Diagrams to illustrate how the front-ends (web and mobile), backend microservices, and external systems collaborate over time to fulfill the system's operations. Each diagram below includes concrete lifelines for the UI layers, the Kong API Gateway, Node.js/Express.js microservices, a single PostgreSQL instance, and event-streaming platforms. To keep the diagrams consistent:

- **UI lifelines** use the role-specific surface (React.js Web Application for Organization Coordinators and System Administrators, and a Flutter Mobile Application for Community Members, Donors, and Volunteers).
- **Microservices** are shown individually (User, Organization, Resource, Request, Volunteer, Task, and Notification Services).
- **Cross-service communication** uses synchronous REST APIs for direct requests (solid arrows) and asynchronous domain events via Apache Kafka for decoupled notifications and updates (open/dashed arrows).
- **Control flows** use alt fragments to handle authorization, stock checks, and validation error paths, while batch actions (such as dispatching multiple notifications) are contained within loop fragments.

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

6.2.1 User Registration and Login:

- User submits registration details which are validated by the User Service.
- Account and identity are created in the database and IAM framework.
- User logs in by submitting credentials for authentication via the IAM framework.
- A JWT access token is generated and returned to grant role-based dashboard access.

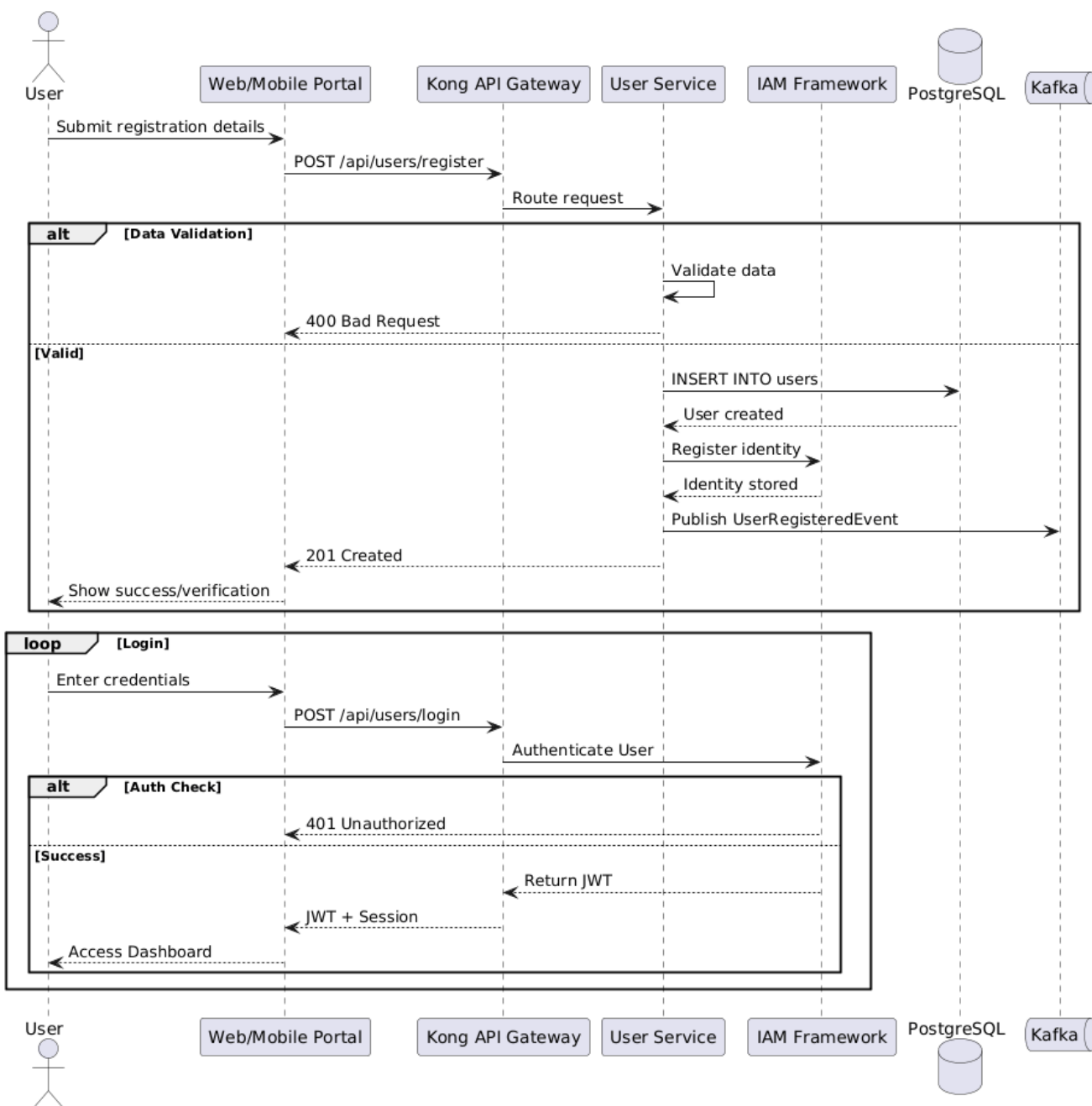


Figure 20 User Registration and Login

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

6.2.2 Community Member Submits a Help Request:

- Community Member submits a resource request via the mobile portal.
- The Request Service validates the input data and roles.
- The request is stored in the database with a 'Pending' status.
- An event is published via Kafka to alert the Organization Coordinator.

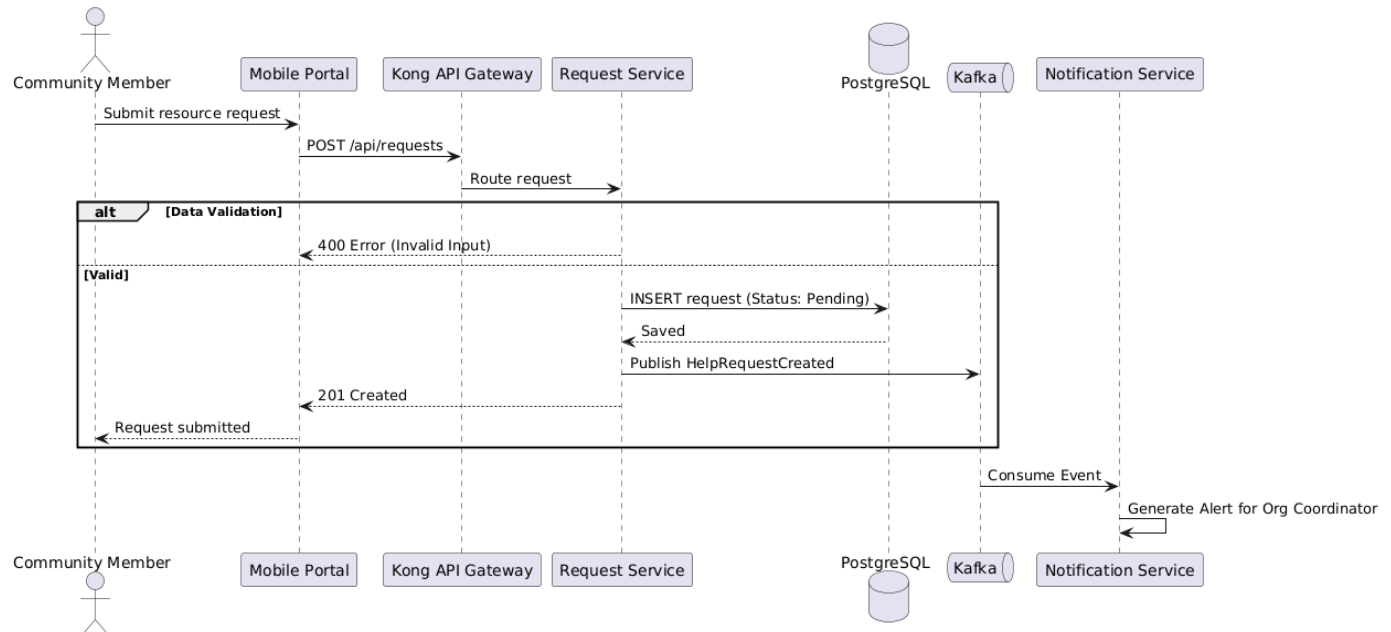


Figure 21 Community Member Submits a Help Request

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

6.2.3 Organization Coordinator Reviews a Help Request:

- Organization Coordinator views pending requests on the web dashboard.
- The coordinator decides to approve or reject the retrieved request details.
- Request Service verifies authorization and updates the request status to 'Approved' or 'Rejected'.
- A notification event is published to inform the requester and potential donors.

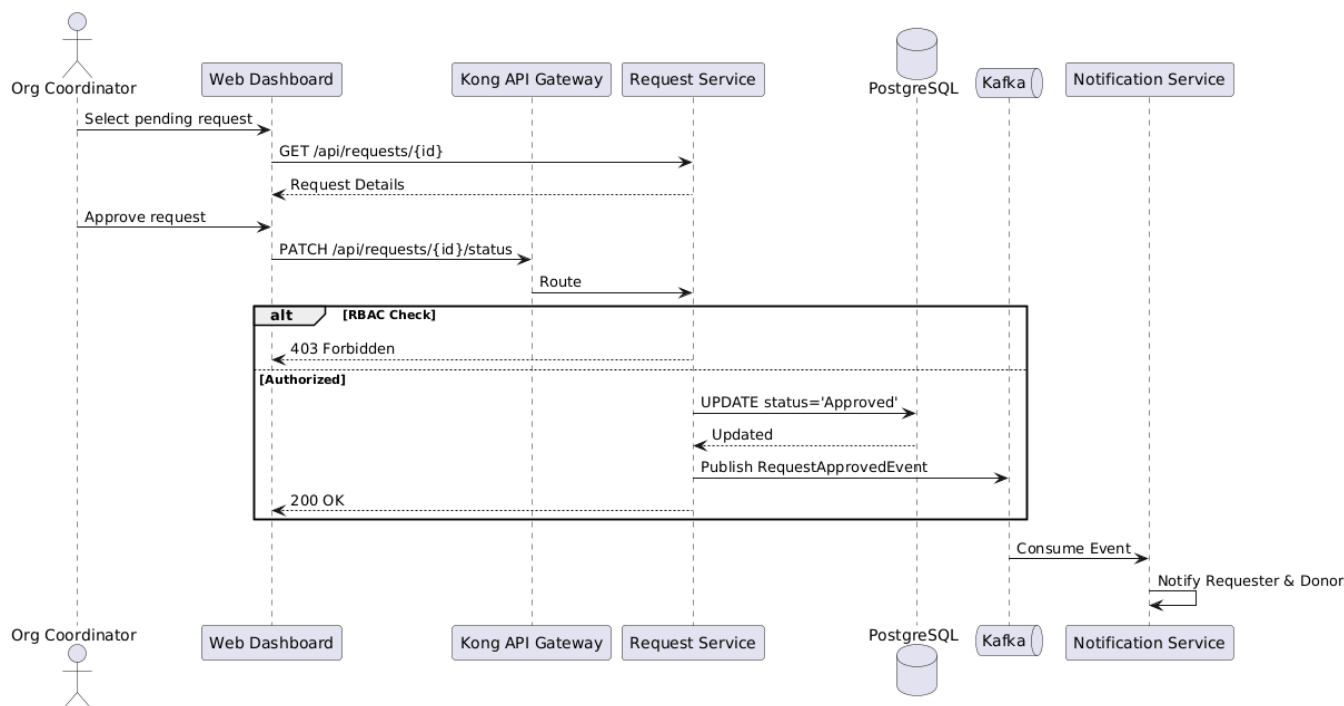


Figure 22 Organization Coordinator Reviews a Help Request

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

6.2.4 Donor Views Approved Needs and Offers a Donation:

- Donor accesses the portal to view a filtered list of 'Approved' requests.
- Donor submits an offer specifying the resource quantity and delivery method.
- Resource Service validates the offer and records the donation with a 'Pending' status.
- An event is triggered to notify the corresponding Organization Coordinator.

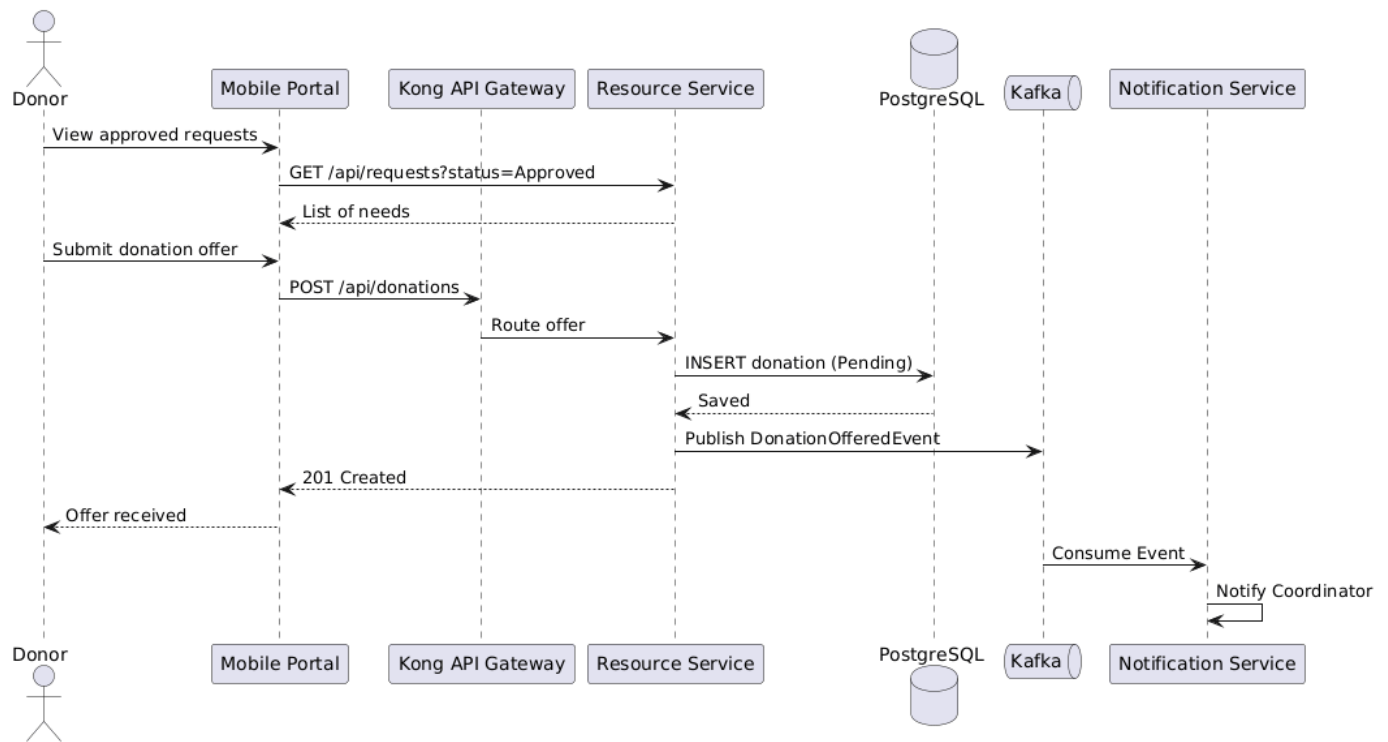


Figure 23 Donor Views Approved Needs and Offers a Donation

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

6.2.5 Organization Coordinator Verifies a Donation:

- Organization Coordinator inspects the details of a pending donation.
- Coordinator accepts or rejects the donation offer.
- Resource Service updates the donation status in the database accordingly.
- Notification Service alerts the Donor regarding the verification decision

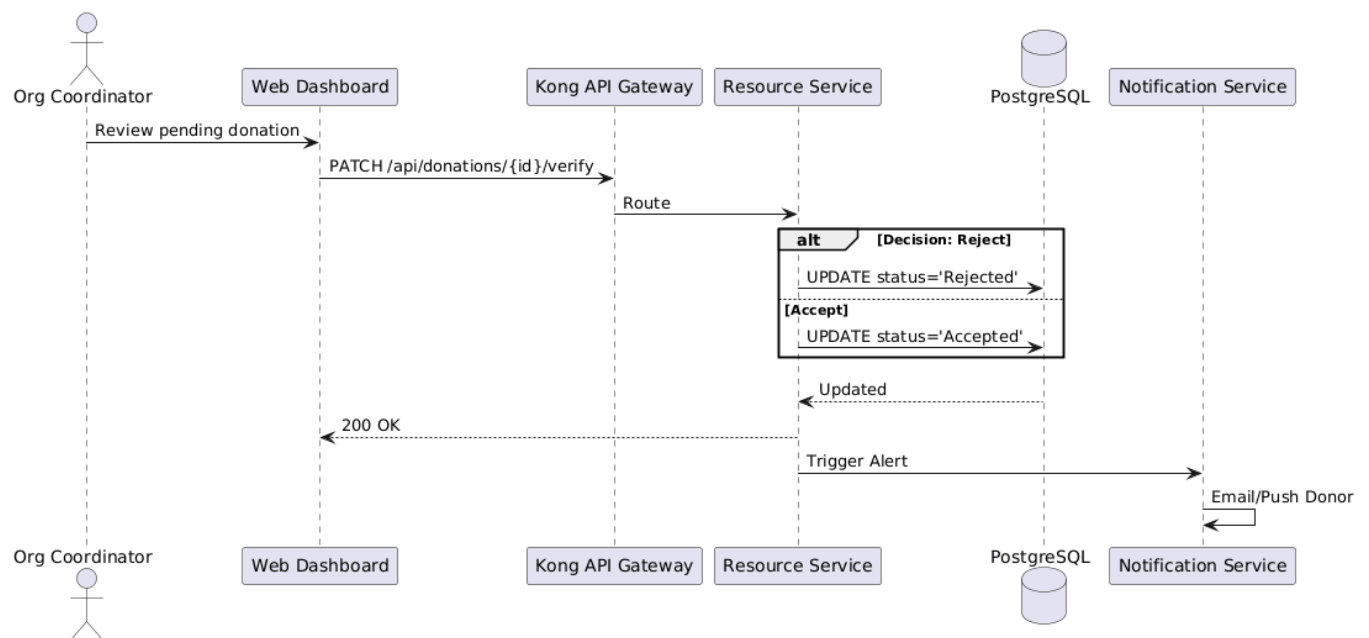


Figure 24 Organization Coordinator Verifies a Donation

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

6.2.6 Donation Receipt and Inventory Update:

- Organization Coordinator or Storekeeper confirms the physical receipt of donated resources.
- Resource Service updates the donation status to 'Received'.
- Inventory stock levels are updated and an inventory transaction is logged in the database.
- An event is published to notify the Donor of the successful receipt.

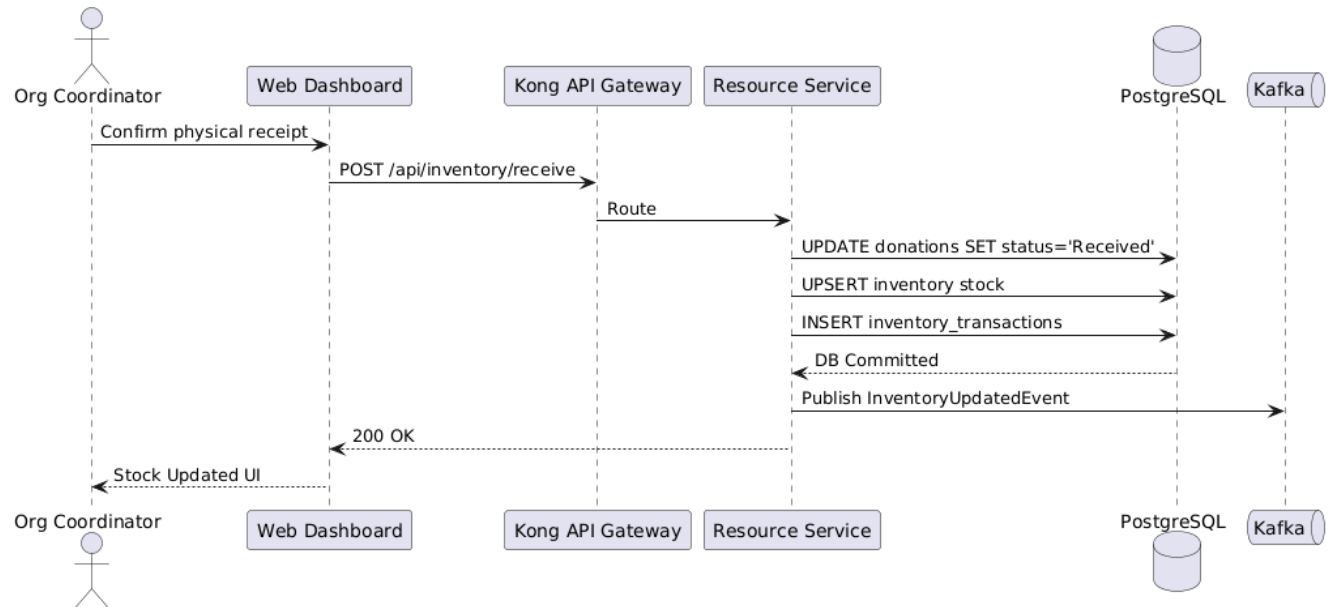


Figure 25 Donation Receipt and Inventory Update

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

6.2.7 Direct Donation Without a Specific Help Request:

- Donor submits a general resource donation offer to a selected organization.
- Resource Service validates the category and quantity of the offer.
- The direct donation is stored in the database as 'Pending'.
- The target organization is notified to review and approve the incoming offer.

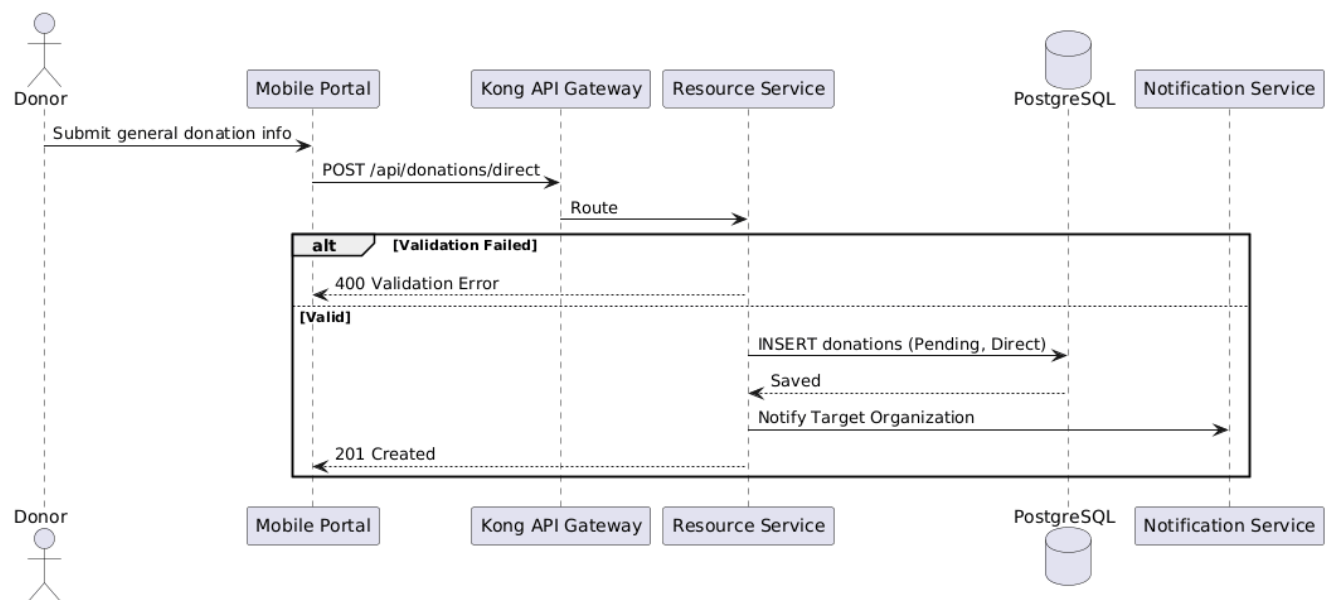


Figure 26 Direct Donation Without a Specific Help Request

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

6.2.8 Resource Allocation to an Approved Request:

- Organization Coordinator selects an approved request to allocate available inventory.
- Resource Service checks the database to verify sufficient stock availability.
- Stock quantities are reserved, and an allocation record is created.
- Request fulfillment status is updated and the community member is notified.

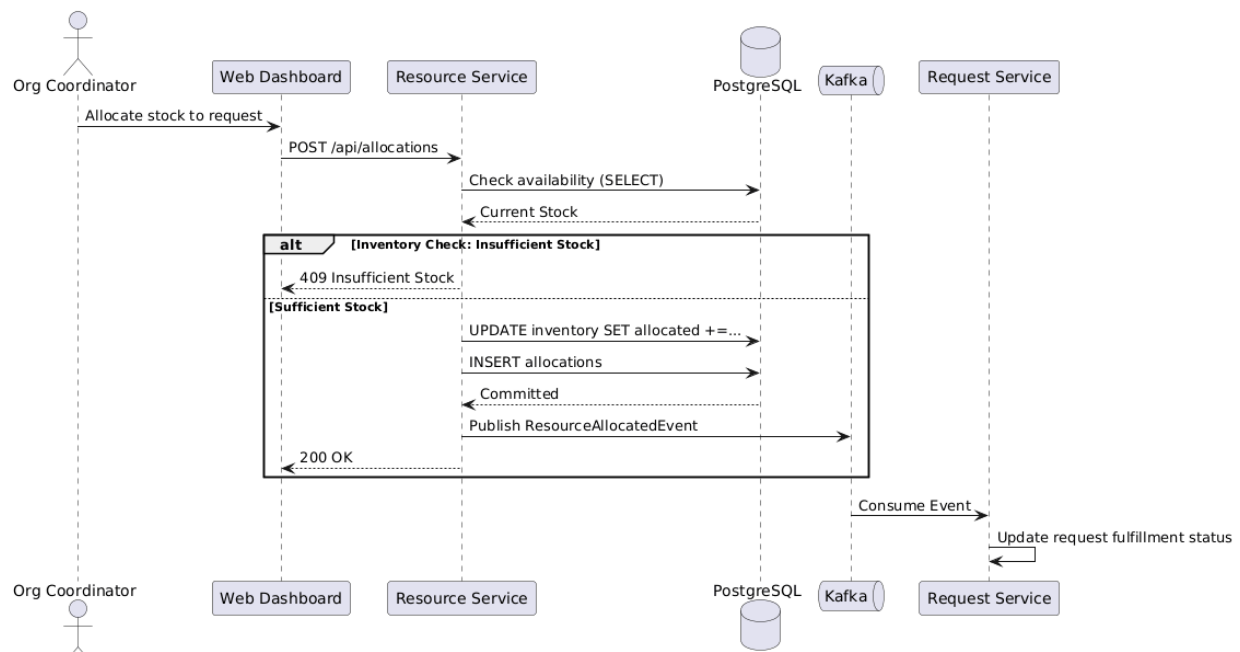


Figure 27 Resource Allocation to an Approved Request

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

6.2.9 Volunteer Registration and Verification:

- Volunteer submits their skills, experience, and availability through the portal.
- Volunteer Service validates the data and stores a 'Pending' profile.
- An alert is sent to the Organization Coordinator for review.
- Once reviewed, the verification status is updated to APPROVED or REJECTED'.

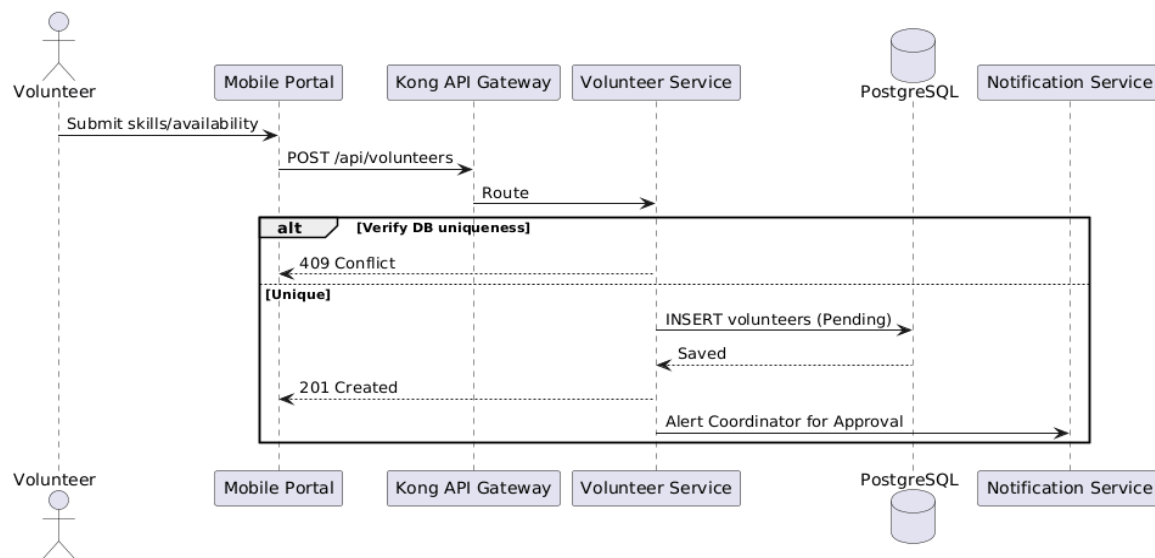


Figure 28 Volunteer Registration and Verification

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

6.2.10 Task Creation and Volunteer Assignment:

- Organization Coordinator creates a disaster response task specifying priority and deadlines.
- Task Service requests available volunteers matching the required skills from the Volunteer Service.
- Coordinator selects a suitable volunteer and confirms the assignment.
- Task and assignment records are created, and the assigned volunteer is notified.

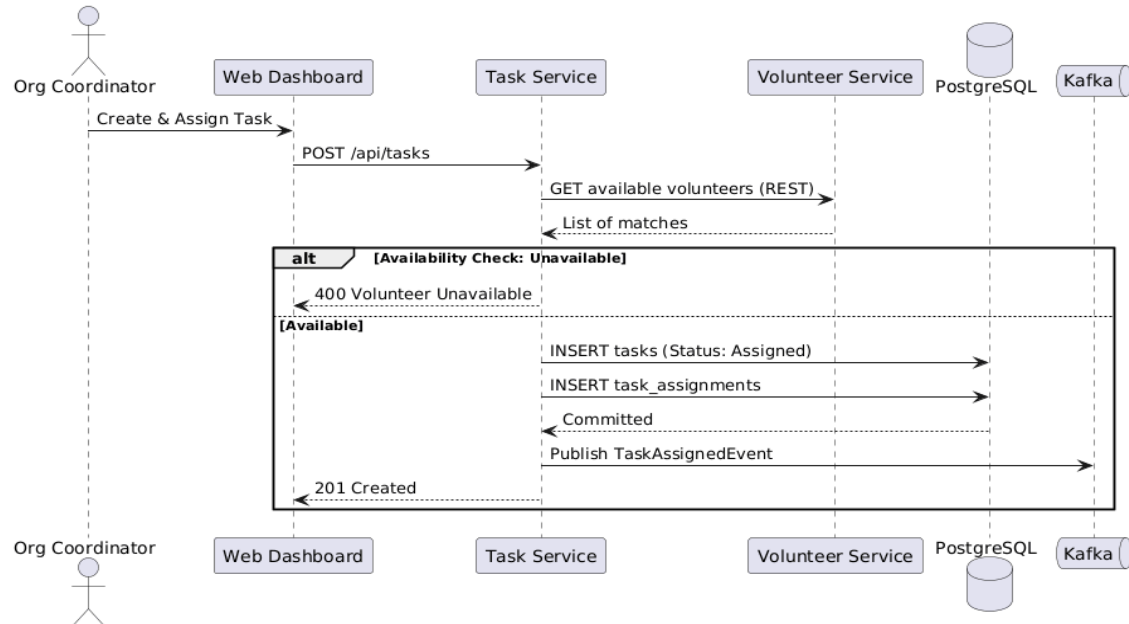


Figure 29 Task Creation and Volunteer Assignment

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

6.2.11 Volunteer Accepts or Rejects a Task:

- Volunteer receives a task notification and reviews the location and deadline.
- Volunteer chooses to either accept or reject the task assignment.
- Task Service updates the assignment status and shifts the task to 'In Progress' if accepted.
- The Organization Coordinator is notified of the volunteer's decision.

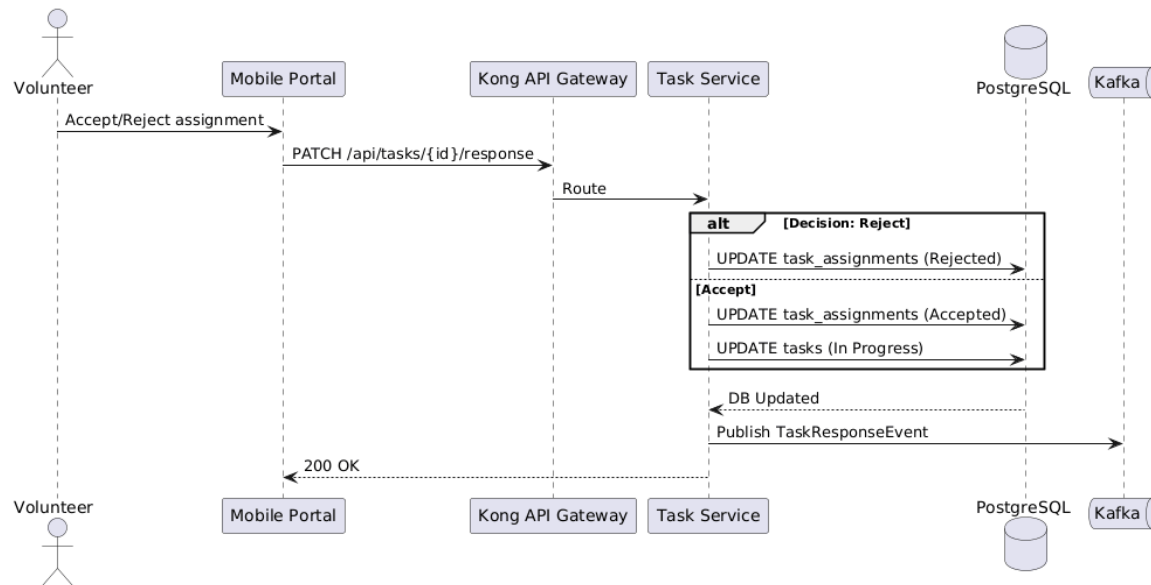


Figure 30 Volunteer Accepts or Rejects a Task

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

6.2.12 Volunteer Updates Task Progress:

- Volunteers submit progress notes for an active task via the mobile portal.
- Task Service validates that the user is the authorized assignee.
- Progress details and timestamps are logged into the database.
- A progress event is published to update the dashboard and notify the coordinator.

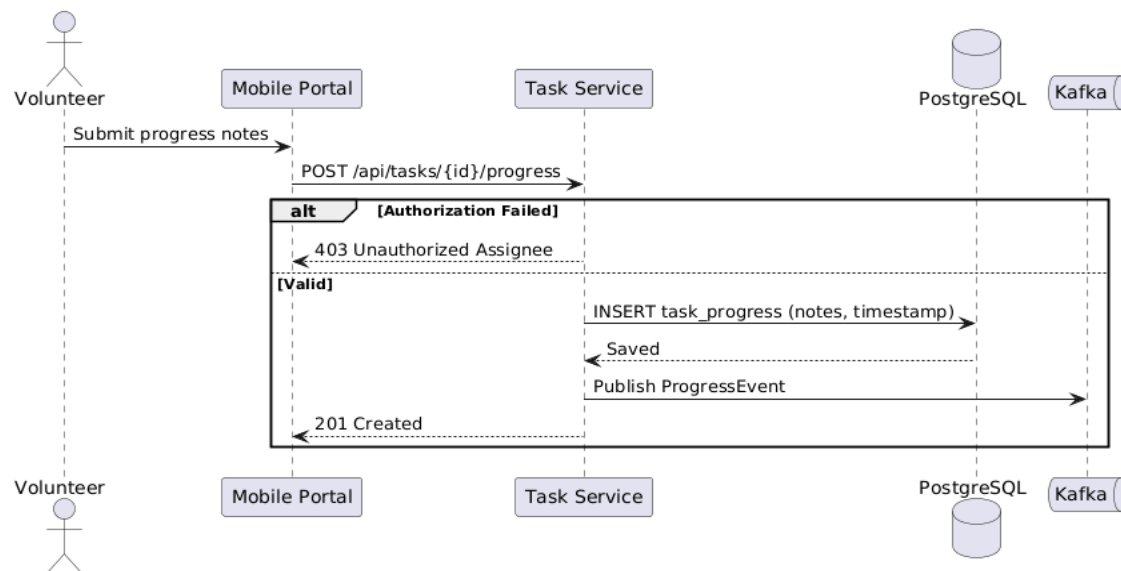


Figure 31 Volunteer Updates Task Progress

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

6.2.13 Task Completion:

- Volunteer marks an active task as completed and provides completion notes.
- Task Service validates the assignment and updates the task status to 'Completed'.
- The completion record is archived in the database.
- A notification event is published to alert the Organization Coordinator.

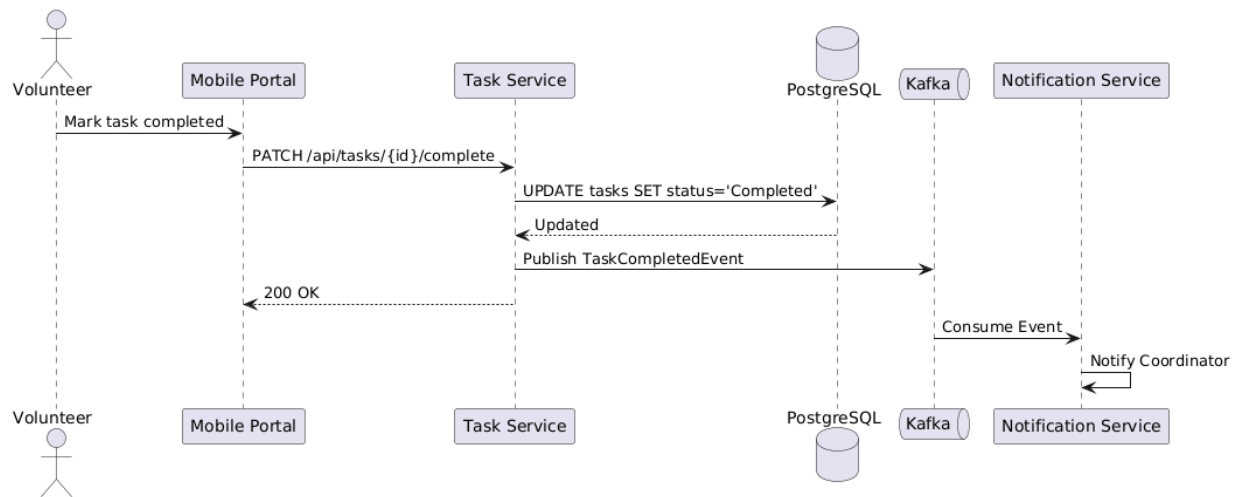


Figure 32 Task Completion

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

6.2.14 Notification Delivery

- A backend microservice publishes a domain event to the Kafka event bus.
- Notification Service consumes the event and fetches user preferences from the database.
- Notifications are dispatched iteratively via Email, Push, or In-App channels.
- Delivery statuses are recorded back into the PostgreSQL database.

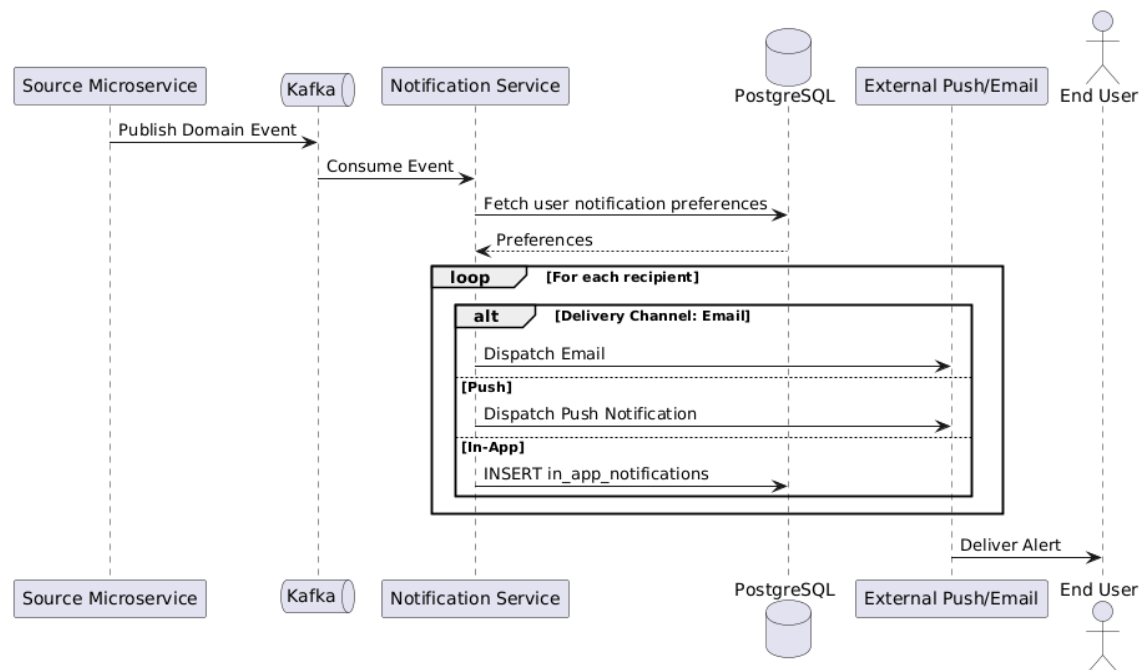


Figure 33 Notification Delivery

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

6.2.15 Organization Registration and Approval

- Organization Coordinator submits organization details and registration documents.
- Organization Service validates the uniqueness of the data and creates a 'Pending' record.
- A System Administrator reviews and approves or rejects the registration.
- The organization status is updated, and the owner receives a notification.

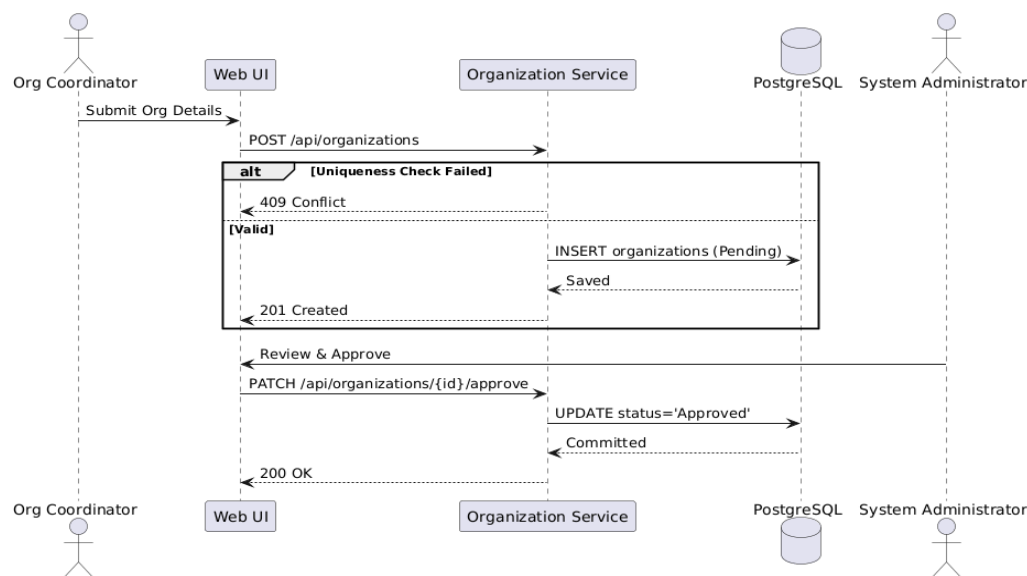


Figure 34 Organization Registration and Approval

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

6.2.16 Organization Membership Management

- Organization Coordinator sends a membership invitation to a user's email.
- Organization Service validates permissions and stores a 'Pending' membership invite.
- Invited user receives the notification and decides to accept or decline.
- The membership status is updated to 'Active' upon acceptance.

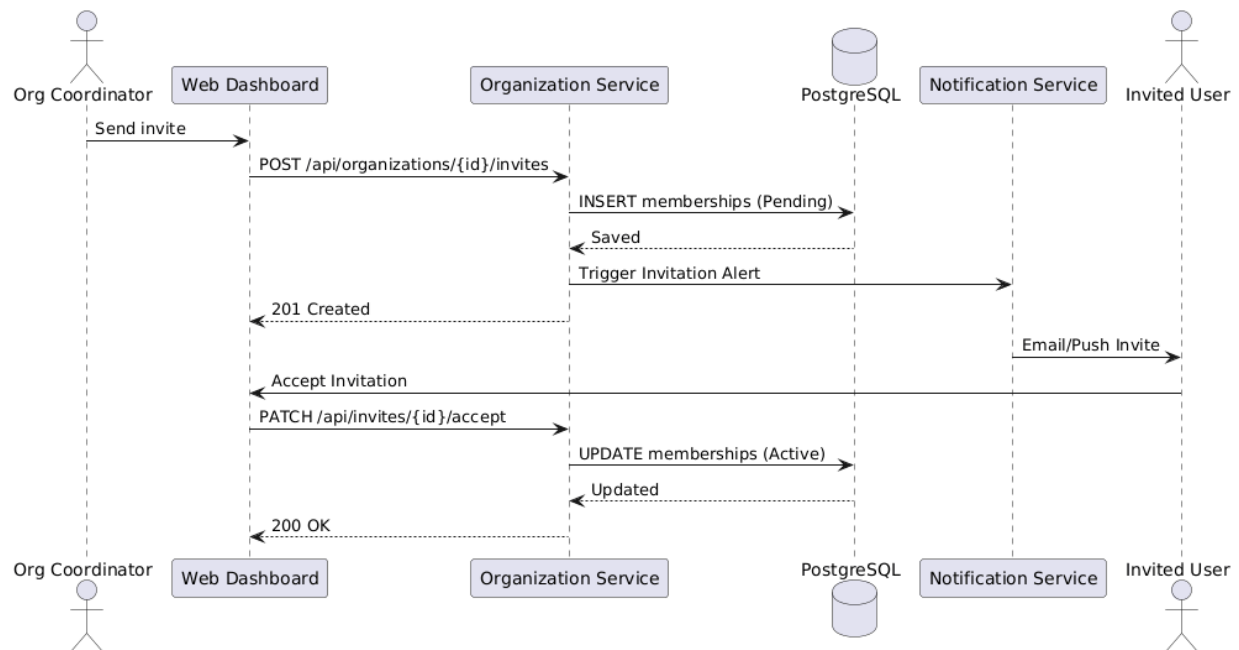


Figure 35 Organization Membership Management

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

7. Deployment View

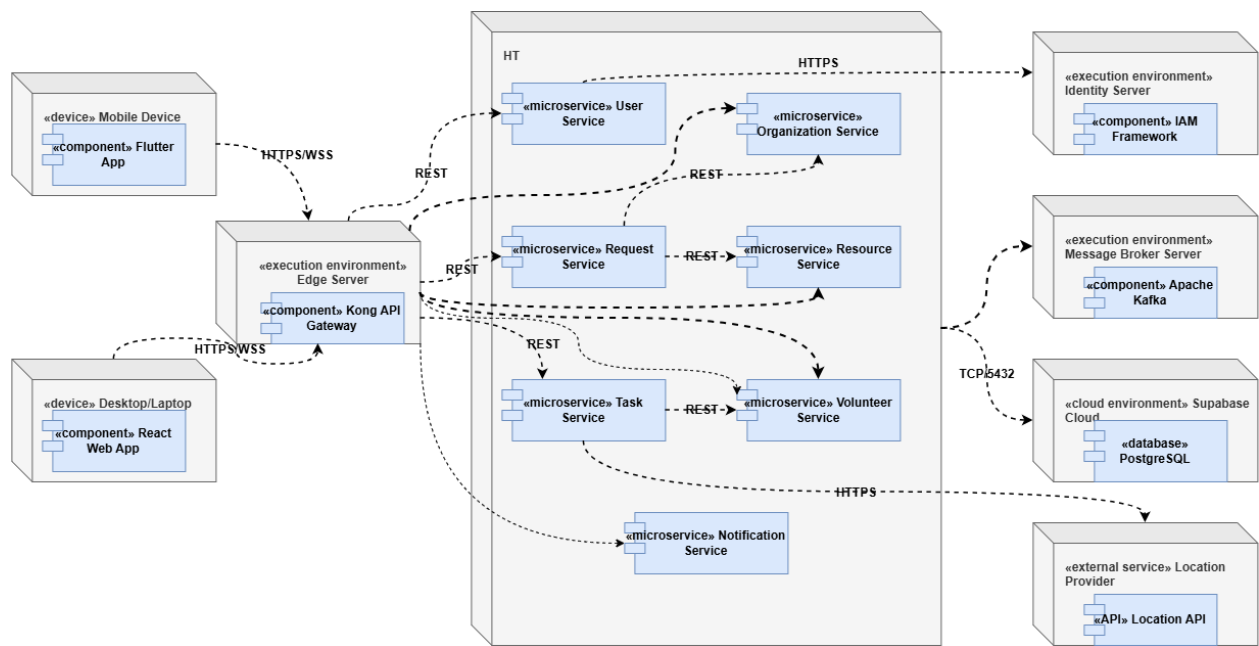


Figure 36 Deployment View

8. Implementation View

This section describes the overall structure of the implementation model, the decomposition of the software into layers and subsystems in the implementation model, and any architecturally significant components of the Resource Coordination for Community Resilience Platform.

8.1 Overview

The Resource Coordination Platform follows a modern microservices architecture pattern combined with a multi-tenant approach. The implementation view foxes on the organization and communication between different architectural layers and services that comprise the entire system. The system is designed to support high scalability, maintainability and technology diversity to serve different user types through appropriate interfaces.

8.2 Layers

The system implementation is divided into several layers based on their responsibilities. Each layer contains related software components and communicates with other layers through defined interfaces.

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

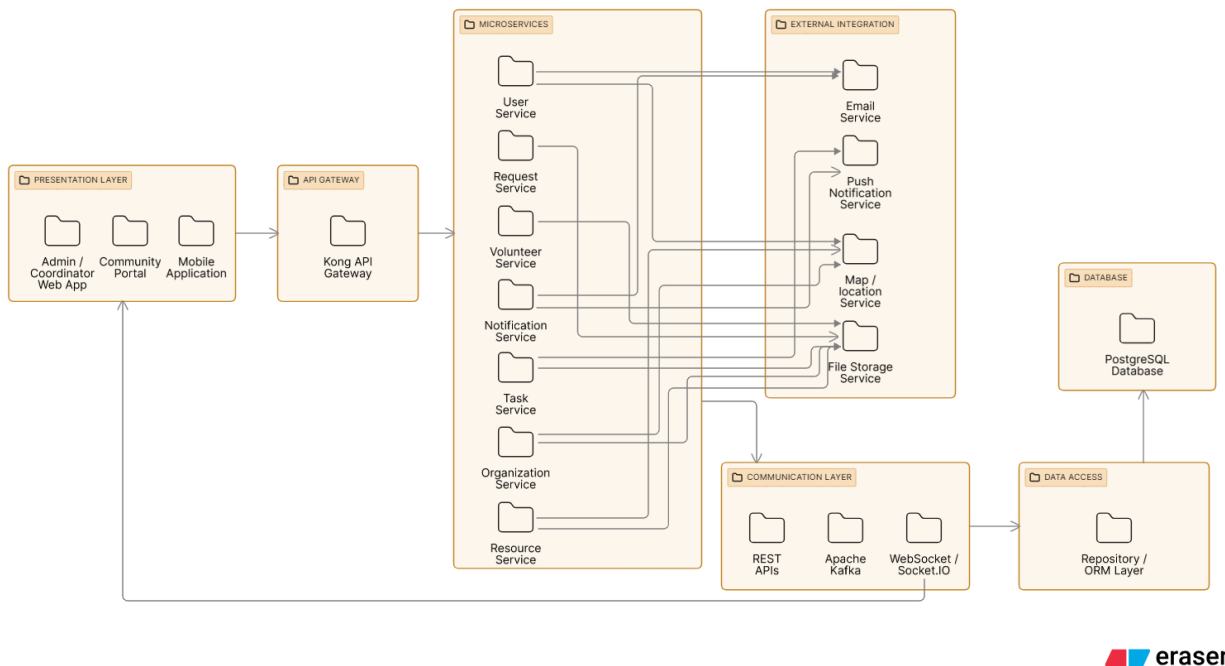


Figure 37 Package Diagram

8.2.1 Presentation Layer

The Presentation Layer contains the web application and mobile application.

- **Web Application:** Provides dashboards and management functions for organization administrators and coordinators.
- **Mobile Application:** Allows users to submit help requests, offer resources, track their activities, and receive notifications. It also allows approved volunteers to manage assigned tasks.

8.2.2 API Gateway Layer

The API Gateway Layer uses Kong as the centralized entry point for backend services.

- Routes client requests to the appropriate microservice.
- Supports authentication and access control.
- Provides centralized API access.

8.2.3 Microservices Layer

The Microservices Layer contains the core business services.

- User and Authentication Service
- Organization Service
- Request Service
- Donation Service
- Inventory Service
- Volunteer and Task Service
- Notification Service

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

8.2.4 External Integration Layer

The External Integration Layer connects the system with third-party platforms.

- Email Service
- Push Notification Service
- Map Service
- File Storage Service

8.2.5 Communication Layer

The Communication Layer manages communication between system components.

- REST APIs support synchronous communication.
- Kafka supports asynchronous communication between microservices.
- WebSockets support real-time notifications and system updates.

8.2.6 Data Access Layer

The Data Access Layer manages communication between the microservices and the database.

- Repository Layer
- ORM Layer
- Database transactions and queries

8.2.7 Data Layer

The Data Layer uses a shared PostgreSQL database to store system information.

- User and organization data
- Help-request data
- Donation and inventory data
- Volunteer and task data
- Notification and audit information

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

9. Data View (optional)

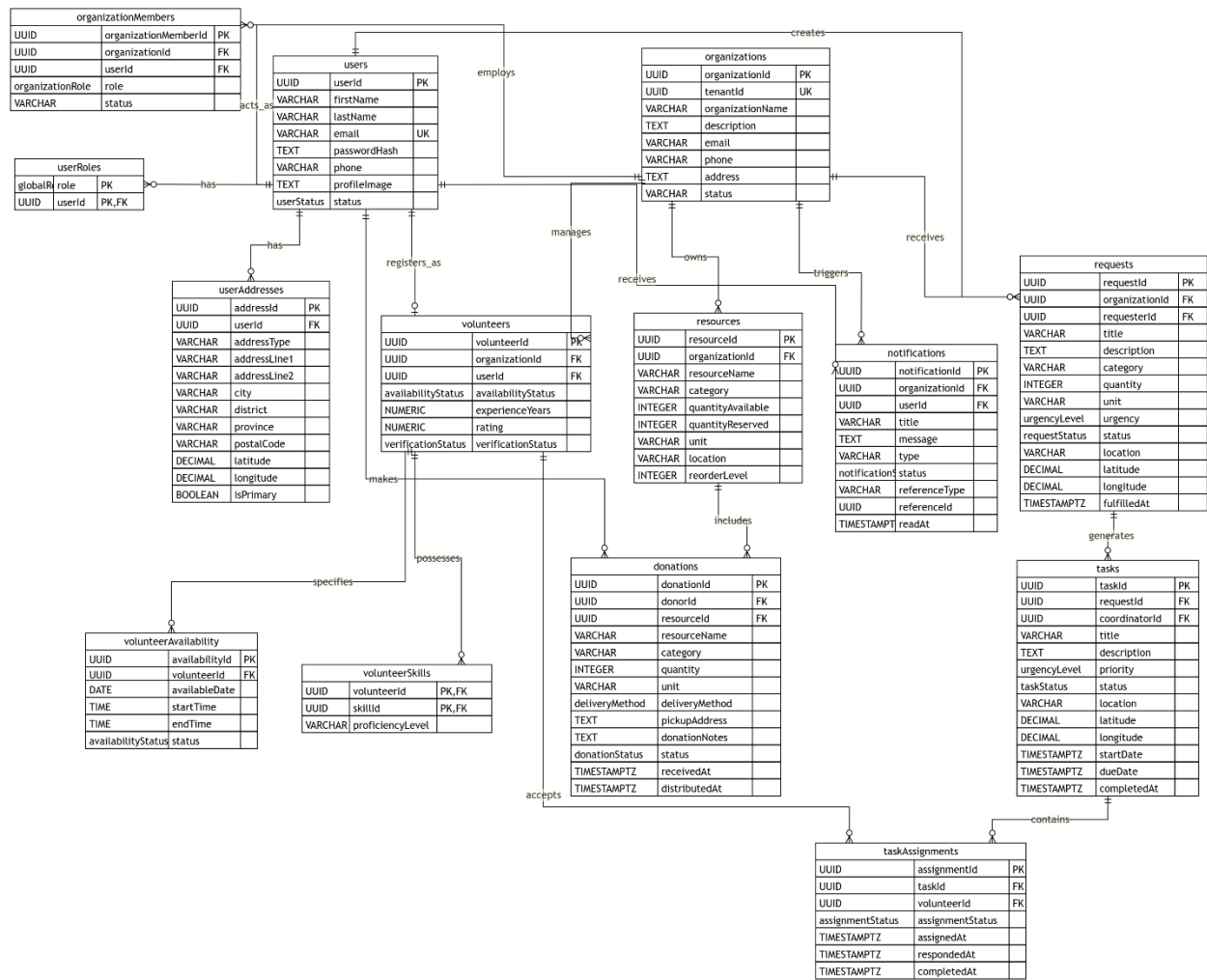


Figure 38 Database Schema

Resource Coordination Platform for Community Resilience	Version: 1.0
Software Architecture Document	Date: 09/08/2026
Resource Coordination Platform - SAD	

10. Size and Performance

System Size:

- System supports multiple organizations and their members
- Users can submit multiple help requests and donation offers
- System maintains records

Code Optimization:

- Database queries are optimized using appropriate indexes and efficient queries,
 - Pagination is used when retrieving large requests, donations and notifications.
 - Efficient algorithms and well structured microservices.
 - Asynchronous communication between services to reduce delays in processing background events.
- These will ensure efficient performance and enhance user experience.

11. Quality

The architecture is designed to provide the system with the required quality attributes while maintaining a maintainable and scalable structure.

Usability:

The system provides simple and role based interfaces for community members, donors, volunteers, coordinators and administrators. The mobile application provides an easy way to request help, offer donations and manage volunteer activities, while the web application provides centralized interfaces for administrative and coordination tasks. Responsive design and clear navigation help users access the system across different devices and environments.

Portability:

The web application can be accessed through standard web browsers, while the mobile application is designed to support mobile platforms. Containerized backend services also provide a consistent environment across development and deployment environments.

Maintainability:

The system is divided into clearly defined layers and independent microservices, making the code easier to understand, test, modify and maintain. Separation between presentation, API gateway, business services, communication, data access and database layers reduces dependencies between components.

12. References

- [1] Betke, H., et al., "A Design Theory for Spontaneous Volunteer Coordination Systems in Disaster Response," Proceedings of the Hawaii International Conference on System Sciences, 2024. Available: <https://scholarspace.manoa.hawaii.edu/items/6e1f845c-7e16-492b-b48d-c4a6bd4f09e5> (Accessed: 02-Jul-2026)
- [2] D. A. Carpenter et al., "Resource exchange patterns between Voluntary Organizations Active in Disaster (VOADs): A multilevel network assessment to improve disaster response capacity," International Journal of Disaster Risk Reduction, vol. 108, 2024.
- [3] M. Sperlinga ., "Decision support for disaster relief: Coordinating spontaneous volunteers," European Journal of Operational Research, vol. 299, no. 2, pp. 690-705, 2022
- [4] Ushahidi, "Crowdsourcing Solutions to Empower Communities." Available: <https://www.ushahidi.com/> (Accessed: 01-Jul-2026).
- [5] Sahana Software Foundation, "Sahana Eden Disaster Management Platform." Available: <https://sahanafoundation.org/eden/> (Accessed: 01-Jul-2026).
- [6] Crisis Cleanup, "Crisis Cleanup Platform." Available: <https://api.crisiscleanup.org/> (Accessed: 01-Jul-2026).